Universidad Andrés Bello
Facultad de Ingeniería · Ingeniería Civil en Informática
Desarrollo Web y Móvil — Evaluación de la Unidad 2

Diseño e implementación de un backend seguro completo

Caso 32: Tienda de vinos “Viña Urbana” – vinos boutique

Integrantes: Stefano Negroni, Isabella Quintero
Repositorio: github.com/Negroni-7/Desarrollo-Web · carpeta Solemne_2
Lunes 09 de noviembre de 2026

Índice

Resumen ejecutivo **4**
Alcance y metodología **4**
Alcance 4
Metodología 4
0. Punto de partida **6**
0.1 Resumen de la solución 6
0.2 Requisitos del caso 6
0.3 Datos del frontend usados por el backend 7
0.4 Pantallas y brechas 7
Módulo 1 — Servicios derivados del frontend **9**
1.1 Catálogo de servicios y trazabilidad 9
1.2 Ejemplos de contrato (datos ficticios) 12
Módulo 2 — Clases y cardinalidades del dominio **15**
2.1 Leyenda de notación 15
2.2 Diagrama de dominio 15
2.3 Relaciones, multiplicidades y reglas 15
2.4 Servicios públicos y DTO 16
2.5 Decisiones del equipo 16
Módulo 3 — Sistemas e integraciones **18**
3.1 Matriz de sistemas 18
3.2 Mapa de componentes y despliegue 20
3.3 Secuencia representativa: compra y pago 21
3.4 Intercambio entre sistemas 21
Módulo 4 — Stakeholders y permisos HTTP **24**
4.1 Registro de stakeholders 24
4.2 Matriz de permisos por servicio, ruta y verbo 24
4.3 Verbos por recurso 26
Módulo 5 — Clases del API Gateway **27**
5.1 Cuántos gateways y por qué 27
5.2 Diagrama de clases del gateway y sus políticas 27
5.3 Qué valida cada capa 28
5.4 Fronteras de confianza 28
5.5 Recorrido de una solicitud autorizada y una denegada 28
Módulo 6 — Rutas y secretos de integración **29**
6.1 Matriz de rutas REST 29
6.2 Operaciones GraphQL 30
6.3 Métodos gRPC 31
6.4 Credencial del usuario vs. credencial entre sistemas 31
6.5 Inventario simbólico de secretos e identidades 32
Módulo 7 — Datos sensibles y secretos del cliente **34**
7.1 Matriz de datos y credenciales 34
7.2 Decisión sobre secretos del cliente 35
7.3 Recorrido de un dato restringido: la dirección de entrega 35
7.4 Sesiones y tokens 35
7.5 Casos de validación ejecutados 35
Módulo 8 — Separación privada y agregaciones GraphQL **37**
8.1 Mapa de clasificación de servicios 37
8.2 Schema 37

8.3 Resolvers y controles . . . 40
8.4 Consulta de prueba y respuesta . . . 41
8.5 Matriz de autorización GraphQL . . . 41
8.6 Evidencia de la agregación ejecutada a través del API Gateway . . . 41
Módulo 9 — Analítica y comunicación gRPC . . . 43
9.1 Caracterización de la carga . . . 43
9.2 ADR-01: gRPC entre orders e inventory . . . 43
9.3 ADR-02: servicio de analítica . . . 44
9.4 Prueba de carga (medición) . . . 44
9.5 Pruebas gRPC y de analítica . . . 45
Módulo 10 — Backend completo e integración verificable . . . 46
10.1 Qué se implementó . . . 46
10.2 Persistencia . . . 46
10.3 Despliegue y operación . . . 47
10.4 Pruebas automatizadas . . . 47
10.5 Matriz requisito | pantalla | servicio | prueba | resultado | evidencia . . . 48
10.6 Ejemplo: consulta protegida autorizada y denegada . . . 48
10.7 Demostración de los flujos principales . . . 48
Conclusiones . . . 52
Resultados . . . 52
Decisiones principales . . . 52
Limitaciones y trabajo futuro . . . 52
Anexos . . . 53
A. Archivos de la entrega . . . 53
B. Referencias . . . 53

Resumen ejecutivo

El presente informe documenta el diseño, la implementación y la validación del backend del caso **Viña Urbana**, comercio de vinos boutique de productores artesanales que ofrece, además, degustaciones con maridaje. El trabajo corresponde a la Evaluación de la Unidad 2 y se estructura según los diez módulos definidos en la guía.

La solución adopta una arquitectura de microservicios privados expuestos a través de un único **API Gateway** con TLS, que concentra la autenticación (introspección de sesiones emitidas por un **Auth Service**), la autorización por ruta y rol, la protección CSRF, la limitación de solicitudes y el manejo controlado de errores. Detrás del gateway operan siete servicios con propiedad exclusiva de sus datos (inventory, orders, clients, payments, tastings, analytics y auth), cada uno con base de datos MongoDB y usuario propios. Las credenciales entre componentes se gestionan en **HashiCorp Vault** con una credencial distinta por tramo y una política de mínimo privilegio por servicio. La comunicación orders → inventory se implementa con **gRPC unary sobre mTLS**, y las vistas agregadas se resuelven mediante un **GraphQL Gateway privado** con autorización por operación y por campo, límites de profundidad y costo, y tratamiento de errores parciales.

Por indicación del docente, la entrega corresponde exclusivamente al backend construido **con los datos del frontend** desarrollado previamente (versión 1.0), sin integrar ni modificar dicho frontend. Los datos del catálogo, los paquetes de degustación, las opciones de los formularios y las reglas publicadas se extrajeron de forma automatizada del `index.html` original y constituyen la base de datos inicial y la referencia de los contratos.

Indicador	**Resultado**
Requisitos funcionales del caso	13 de 13 implementados (R01-R13)
Servicios expuestos	34 operaciones (S01-S34): 31 rutas REST y 3 operaciones GraphQL
Sistemas privados	7 microservicios, GraphQL Gateway y proveedor de pagos de prueba
Pruebas automatizadas	97 de 97 aprobadas (contratos, datos del frontend, seguridad, flujos, GraphQL, resiliencia y gRPC)
Casos de validación de seguridad (módulo 7)	V1-V6 verificados en ejecución
Demostración de flujos vía API	49 de 49 pasos con el resultado esperado
Prueba de carga	5 de 5 escenarios cumplen el objetivo de p95, con 0 % de errores

Alcance y metodología

Alcance

- Se implementan todos los requisitos funcionales del caso y las operaciones requeridas por las pantallas del frontend v1.0, incluidas las funciones que aún carecen de pantalla (brechas B1-B6), las cuales quedan resueltas a nivel de backend.
- Se excluye la integración visual del frontend, de acuerdo con la restricción establecida por el docente. Los flujos se demuestran mediante un cliente HTTP (script de demostración y colección Postman).
- La pasarela de pagos se reemplaza por un adaptador de prueba que conserva el contrato de una pasarela real (credenciales de comercio, página de pago, notificación firmada y consulta servidor a servidor). Se incluye un adaptador para el ambiente de integración de Transbank Webpay Plus, no ejecutado por ausencia de salida a Internet en el entorno de pruebas.

Metodología

- **Trazabilidad:** cada decisión se vincula con un requisito (R), una pantalla (P) o brecha (B) y un servicio (S). Los mismos identificadores se emplean en el código (`routes.py`) y en los nombres de las pruebas automatizadas.

- **Diseño:** se aplicaron criterios de cohesión, acoplamiento y propiedad de datos para delimitar los sistemas; las decisiones sobre analítica y gRPC se registran en ADR (módulo 9).
- **Seguridad:** se siguieron las recomendaciones de OWASP (REST Security, Authorization y Secrets Management) y la documentación oficial de GraphQL y gRPC.
- **Validación:** toda afirmación de seguridad o de comportamiento se respalda con una prueba ejecutada contra el sistema en funcionamiento; las evidencias se conservan en `docs/evidencias/` .
- **Reproducibilidad:** el despliegue se define en `docker-compose.yml` y existe un modo de ejecución local equivalente (`scripts/run_local.py`). El procedimiento completo de instalación, ejecución y prueba se documenta en `README.md`.

0. Punto de partida

Esta sección establece la línea base del trabajo: los requisitos funcionales derivados del caso, los datos del frontend v1.0 reutilizados por el backend y las funciones del negocio que no cuentan con pantalla (brechas), junto con su resolución en esta entrega.

0.1 Resumen de la solución

Se implementó un backend de microservicios privados detrás de un único **API Gateway** público. Un cliente (el frontend en el futuro, Postman o las pruebas) sólo conoce `https://localhost:8443/api/v1`; todo lo demás (siete microservicios, el GraphQL Gateway, MongoDB y Vault) vive en redes internas. Cada tramo entre componentes usa TLS y una credencial propia guardada en Vault; orders e inventory se comunican por **gRPC con mTLS**; el **GraphQL Gateway** privado agrega orders + inventory + payments (pedidos del cliente) y analytics + inventory (panel de ventas). El stack respeta las tecnologías del curso: Python/FastAPI, Node/Apollo, MongoDB y HashiCorp Vault, con despliegue reproducible en Docker Compose.

Componente	**Tecnología**	**Responsabilidad**
API Gateway	Python 3.12 · FastAPI · uvicorn TLS	Rutas explícitas, autenticación, autorización por rol, CSRF, límites, errores controlados
Auth Service	FastAPI · Argon2id · JWT HS256	Credenciales, sesiones revocables, introspección
clients · inventory · orders · payments · tastings · analytics	FastAPI · pymongo	Servicios de negocio, cada uno dueño de su base MongoDB
gRPC inventory	grpcio · protobuf · mTLS	Cotizar, reservar, confirmar y liberar stock
GraphQL Gateway	Node 22 · Apollo Server 5 · DataLoader	Agregaciones privadas con autorización por operación y campo
Persistencia	MongoDB 8 (TLS, un usuario por base)	vu_auth, vu_clients, vu_inventory, vu_orders, vu_payments, vu_tastings, vu_analytics
Secretos	HashiCorp Vault (KV v2)	Una política y un token por servicio; secretos aleatorios generados por vault-init
Proveedor de pagos	Sandbox propio (adaptador de prueba)	Simula una pasarela tipo Webpay; adaptador Transbank incluido

0.2 Requisitos del caso

Los requisitos provienen del caso y de las pantallas del frontend v1.0. Se separan de las decisiones de arquitectura, que se justifican en los módulos siguientes.

ID	**Requisito del caso**	**Origen**
R01	Venta exclusiva a mayores de 18 años	Modal de edad y bloque "Edad mínima"
R02	Catálogo de vinos boutique con filtro por tipo (tinto, blanco, rosado, especial)	Sección Catálogo y filtros
R03	Vinos recomendados	Carrusel "Recomendados para ti"
R04	Carrito de compras con subtotal y total	Sidebar del carrito
R05	Despacho a todo Chile; gratis sobre $50.000 en Providencia; 2-5 días hábiles	Información importante
R06	Pago seguro con tarjeta (Visa, Mastercard, Amex)	Información importante, botón " Ir a Checkout"
R07	Registro e inicio de sesión de clientes	Modal "Mi cuenta"
R08	Reserva de degustaciones: 3 paquetes, fecha, hora, personas y comentarios	Sección Degustaciones
R09	Formulario de contacto	Sección Contáctanos
R10	Información institucional (historia, cifras)	Sección Nosotros

ID	Requisito del caso	Origen
R11	Administración del catálogo, stock y despacho de pedidos	Implícito en la operación de la tienda
R12	Seguimiento de pedidos y boleta de la compra	Implícito en una venta en Chile
R13	Reportes de ventas y reservas para la gerencia	Implícito ("Hoy seleccionamos más de 200 etiquetas...")

0.3 Datos del frontend usados por el backend

Restricción del docente: la entrega de backend se hace con los datos del frontend y no con el frontend integrado. Por eso el frontend v1.0 no se modificó: se usó como fuente de requisitos, de datos iniciales y de forma de los contratos. El script `scripts/extraer_datos_frontend.py` lee `Entrega_Frontend/index.html` y genera `scripts/datos_frontend.json`, del que se cargan la base de datos y se verifican las pruebas D01-D07.

Dato del frontend v1.0	Dónde aparece	Uso en el backend
9 vinos: nombre, tipo, viña, ml, % alcohol, precio, imagen	Tarjetas de #catalogo (data-precio, data-imagen)	Colección products de inventory (S01); el id es el nombre del archivo de imagen
Vinos recomendados (6)	Carrusel #recomendados	Campo recomendado (S01 ?recomendado=true)
Filtros Tintos / Blancos / Rosados / Especiales	Botones de filtro	Enumeración tipo de S01
3 paquetes: nombre, descripción, incluye, duración, precio	Tarjetas de #degustaciones	Colección packages de tastings (S17); duración en minutos
Horas 10:00, 12:00, 16:00, 18:00, 20:00	Formulario de reserva	Horarios válidos de S18/S19
Campos de reserva (nombre, email, teléfono, paquete, fecha, hora, personas, comentarios)	Formulario de reserva	Cuerpo de S19
Asuntos: Consulta sobre vinos, Pedido especial, Degustación privada, Otro	Formulario de contacto	Enumeración asunto de S22
Campos de registro (nombre, teléfono, email, contraseña) y login	Modal Mi Cuenta	Cuerpo de S03 y S04
Carrito: nombre, precio, cantidad, subtotal, total	Sidebar del carrito (app.js)	S02 recibe sólo productoId y cantidad; el servidor calcula precios y totales
Despacho gratis sobre $50.000 en Providencia; 2-5 días	Información importante	Regla de costo de despacho de S02/S10
Venta a mayores de 18 años	Modal de edad e Información importante	Validación de fechaNacimiento en S03
Pago con Visa, Mastercard, Amex	Información importante	Pago en la página del proveedor (S14-S16)

Stock inicial, roles y usuarios no existen en el frontend: los define el backend (declarados en el mismo JSON y en `seed_data.py`).

0.4 Pantallas y brechas

El frontend es responsivo (Bootstrap 5): escritorio y móvil comparten las mismas pantallas con variaciones visuales (menú colapsable, tarjetas en una columna, carrusel de 1 a 4 tarjetas). Ninguna variación visual originó un servicio nuevo.

ID	Pantalla del frontend v1.0	Operaciones que requiere
P01	Verificación de edad (modal)	Aviso de interfaz; la regla R01 se valida en el servidor al registrarse (S03)
P02	Catálogo + filtros	S01
P03	Recomendados (carrusel)	S01 con recomendado=true
P04	Nosotros	Ninguna: contenido estático público
P05	Degustaciones + formulario de reserva	S17, S18, S19

ID	Pantalla del frontend v1.0	Operaciones que requiere
P06	Contacto	S22
P07	Carrito (incluye botón "Ir a Checkout")	S02 y, al presionar Checkout, S10 y S14
P08	Modal Mi Cuenta (Iniciar sesión / Registrarse)	S03, S04, S06
P09	Información importante	Reglas de negocio de S02, S03 y S14

Brecha	Función del caso sin pantalla en v1.0	Resolución en esta entrega (backend)	Propuesta de incorporación al frontend
B1	Checkout: dirección de despacho y confirmación	S02 con destino, S10, S14 implementados y probados	Formulario de despacho tras "Ir a Checkout"
B2	Resultado del pago	S15 y S16; el retorno del proveedor llega a `/?pago=<id>`	Pantalla que llame a verify y muestre la boleta
B3	Área del cliente: pedidos, reservas, perfil, cerrar sesión	S05, S07-S09, S11-S13, S20, S21	Pestañas en el modal Mi Cuenta; Mis pedidos vía GraphQL
B4	Administración: catálogo, stock, despacho, reservas, mensajes	S23-S33 con roles ADMIN y SOMMELIER	Panel interno separado
B5	Reportes de ventas	S34 (GraphQL: analytics + inventory)	Sección de analítica del panel
B6	El registro no pide fecha de nacimiento ni aceptación de términos	S03 los exige (regla R01 validada en el servidor)	Agregar ambos campos al formulario de registro

Todas las brechas quedan **resueltas en el backend**: sus servicios están implementados, persisten datos, aplican reglas y permisos, y se demuestran con pruebas automatizadas y con la demostración vía API (`docs/evidencias/demo_api/flujos.md`) y la colección Postman (`docs/postman/`). La incorporación visual queda propuesta, porque el docente indicó no integrar el frontend en esta entrega.

Módulo 1 — Servicios derivados del frontend

El módulo identifica las operaciones que requiere cada pantalla y cada brecha, y define para cada una su contrato: verbo y ruta, parámetros, cuerpo, respuesta, códigos de estado, reglas de validación, clasificación de los datos transportados y nivel de acceso. Como principio rector, ningún dato aportado por el navegador se considera evidencia confiable para precios, totales o estados de pago: el cliente informa identificadores y cantidades, y el servidor calcula o verifica el resto.

Se recorrió cada pantalla del frontend v1.0 (P01-P09) y cada brecha (B1-B6) registrando acciones, formularios, listados y mensajes; los campos de los formularios del frontend definen los cuerpos de los contratos. Las operaciones repetidas en varias pantallas reutilizan el mismo servicio (por ejemplo S01 alimenta el catálogo y el carrusel; S02 alimenta el carrito y el checkout). No se inventaron operaciones destructivas: los "borrados" del negocio (anular pedido, retirar vino, cancelar reserva, eliminar cuenta) conservan el registro y su historial. Lo que el navegador envía nunca se usa como precio, total ni estado de pago: el servidor lo calcula o lo verifica.

Acciones sin servicio (justificadas): **P04 Nosotros** es contenido estático público; **P01** es un aviso de interfaz (la regla R01 se valida en S03); el **movimiento del carrusel** y el **menú móvil** son comportamiento local del navegador. Diferencias deliberadas con el frontend v1.0: el registro exige fecha de nacimiento y términos (B6) y contraseña ≥ 8 con letras y números (el formulario v1 pedía 6); la reserva recibe personas como número 1-12 (el select v1 usa rangos) para poder controlar cupos.

1.1 Catálogo de servicios y trazabilidad

ID	Req.	Pantalla y acción	Propósito	Verbo y ruta	Entrada	Salida	Reglas y errores	Clasif.	Acceso
S01	R02 R03	P02 cargar/ filtrar · P03 carrusel	Consultar catálogo	GET /api/v1/ products	query: tipo (enum), recomendado (bool), pagina (1-100), tamano (1-50). Sin cuerpo	200 {items[{id, nombre, tipo, vina, volumenMl, gradoAlcohol, precio, imagen, recomendado, disponible}], total, pagina, tamano}	400 filtro inválido. Sólo productos activos. No expone stock exacto	Público	Anónimo
S02	R04 R05	P07 abrir carrito · P09 regla de despacho	Cotizar carrito y despacho	POST /api/v1/ orders/quote	body {items[{productoId, cantidad 1-24}] (1-50), destino?{comuna, region}}	200 {items[precioUnitario, subtotal, disponible], noDisponibles[], subtotal, despacho|null, total}	Precio oficial de inventory (gRPC). Despacho: 0 si Providencia y subtotal $\geq$ 50.000; 3.990 RM; 6.990 regiones. 503 si inventory no responde	Público	Anónimo
S03	R01 R07	P08 Registrarse · P01/P09 edad	Registrar cliente	POST /api/v1/ auth/register	body {nombre, email, password $\geq$8 letras+números, telefono?, fechaNacimiento, aceptaTerminos}	201 {userId, email, nombre}	422 UNDERAGE (<18). 409 EMAIL_IN_USE. 400 validación. Contraseña → Argon2id. Perfil en clients (compensación si falla)	Restringido + credencial	Anónimo (límite 10/ min IP)

ID	Req.	Pantalla y acción	Propósito	Verbo y ruta	Entrada	Salida	Reglas y errores	Clasif.	Acceso
S04	R07	P08 Iniciar sesión	Iniciar sesión	POST /api/v1/ auth/login	body {email, password}	200 {user{id,email,rol}, expiresIn} + Set-Cookie vu_session (HttpOnly, Secure, SameSite=Strict)	401 genérico (no revela si existe). 429 tras 5 fallos (bloqueo 15 min). El token no viaja en el cuerpo	Credencial	Anónimo (límite 20/ min IP)
S05	R07	B3 cerrar sesión	Cerrar sesión	POST /api/v1/ auth/logout	cookie de sesión	200 {sesionCerrada: true} + borra cookie	Revoca el jti en Auth: el token deja de servir aunque se haya copiado	Credencial	Autenticado (todos)
S06	R07	P08 estado del botón de usuario	Consultar sesión	GET /api/v1/ auth/session	cookie	200 {userId, email, rol}	401 si expiró o fue revocada	Restringido	Autenticado (todos)
S07	R07	B3 perfil · B1 prellenar	Consultar perfil propio	GET /api/v1/ clients/me	cookie	200 {nombre, email, telefono, fechaNacimiento, direccion}	Sólo el titular (id desde la sesión, nunca desde la URL)	Restringido	CLIENTE
S08	R07 R05	B3 guardar perfil	Modificar perfil	PUT /api/v1/ clients/me	body {nombre, telefono?, direccion?{calle, numero, depto?, comuna, region}}	200 perfil actualizado	PUT reemplaza la representación editable. Email no editable. 400 validación	Restringido	CLIENTE
S09	R07	B3 eliminar cuenta	Eliminar cuenta	DELETE /api/ v1/clients/me	cookie	200 {eliminado:true} + borra cookie	409 si hay pedidos o reservas en curso. Anonimiza perfil y desactiva credencial; pedidos se conservan por respaldo tributario	Restringido	CLIENTE
S10	R04 R05 R06	P07 "Ir a Checkout" → B1	Crear pedido	POST /api/v1/ orders	header Idempotency-Key (uuid). body {items[], entrega{nombreReceptor, calle, numero, depto?, comuna, region, telefono, indicaciones?}} — campos extra prohibidos	201 {id, numero, estado: PENDIENTE_PAGO, items, subtotal, despacho, total, venceEn} · 200 si se repite la clave	Precios del servidor; reserva stock por gRPC; 409 PRODUCTS_UNAVAILABLE; 503 si inventory no responde (no crea pedido); 400 si llega precio/ total	Restringido	CLIENTE
S11	R12	B3 mis pedidos	Mis pedidos con productos y pago	POST /api/v1/ graphql · misPedidos	query GraphQL { misPedidos(pagina, tamano ≤10)}	data.misPedidos {items[ResumenPedido], total}	Sólo pedidos propios. Errores parciales si un subsistema cae	Restringido	CLIENTE
S12	R12	B3 detalle de pedido	Resumen de un pedido	POST /api/v1/ graphql · resumenPedido	{resumenPedido(id)}	data.resumenPedido o null	Dueño o ADMIN; si es ajeno responde null (no revela existencia)	Restringido	CLIENTE, ADMIN
S13	R12	B3 anular	Anular pedido pendiente	POST /api/v1/ orders/{id}/cancel	path id	200 pedido CANCELADO	Sólo PENDIENTE_PAGO y propio (404 si ajeno). Libera stock. No borra el registro. 409 otros estados	Restringido	CLIENTE
S14	R06	P07 Checkout · P09 pago seguro → B1	Iniciar pago	POST /api/v1/ payments	body {pedidoId} (monto prohibido)	201 {pagoId, estado: INICIADO, monto, redirectUrl}	Monto desde orders. 404 pedido ajeno. 409 si no está pendiente. Reutiliza pago INICIADO	Restringido	CLIENTE

ID	Req.	Pantalla y acción	Propósito	Verbo y ruta	Entrada	Salida	Reglas y errores	Clasif.	Acceso
S15	R06 R12	B2 resultado del pago	Verificar pago	POST /api/v1/ payments/{id}/ verify	path id	200 {pagoId, pedidoId, estado, monto, boleta?}	El servidor consulta al proveedor (servidor a servidor). El navegador no puede marcar pagado	Restringido	CLIENTE
S16	R06	— (proveedor)	Notificación del proveedor	POST /api/v1/ payments/ webhook	cuerpo JSON {event_id, token, status, amount} + X-Provider-Timestamp + X-Provider-Signature	200 {recibido, duplicado}	401 firma inválida o vencida (>5 min). event_id repetido no repite efectos	Restringido	Identidad del proveedor (HMAC)
S17	R08	P05 tarjetas de paquetes	Consultar paquetes	GET /api/v1/ tastings/packages	sin parámetros	200 {items[{id, nombre, descripcion, incluye[], duracionMin, precioPorPersona, imagen}]}	—	Público	Anónimo
S18	R08	P05 fecha y hora preferida	Consultar cupos	GET /api/v1/ tastings/ availability	query fecha (AAAA-MM-DD)	200 {fecha, capacidad, horarios[{hora, cuposDisponibles}]}	422 fecha pasada, > 90 días o domingo	Público	Anónimo
S19	R08	P05 Confirmar Reserva	Reservar degustación	POST /api/v1/ tastings/ reservations	body {paqueteId, fecha, hora, personas 1-12, contacto{nombre, email, telefono}, comentarios?}	201 {id, paquete, fecha, hora, personas, total, estado:CONFIRMADA}	Cupo atómico (capacidad 12). 409 NO_CAPACITY. Total = precio × personas en servidor. Pago en el local	Restringido + sensible (comentarios)	CLIENTE
S20	R08	B3 mis reservas	Mis reservas	GET /api/v1/ tastings/ reservations/me	cookie	200 {items[...]}	Sólo propias	Restringido	CLIENTE
S21	R08	B3 cancelar reserva	Cancelar reserva	POST /api/v1/ tastings/ reservations/{id}/ cancel	path id	200 reserva CANCELADA	404 ajena; 409 con < 24 h de anticipación o ya gestionada; libera cupos	Restringido	CLIENTE
S22	R09	P06 Enviar Mensaje	Enviar mensaje de contacto	POST /api/v1/ contact	body {nombre, email, asunto (enum), mensaje 5-2000}	201 {id, recibidoEn, estado:PENDIENTE}	Límite 5/min por IP; retención 12 meses	Restringido	Anónimo
S23	R11	B4 inventario	Listar inventario	GET /api/v1/ admin/products	—	200 {items[+stock, activo]}	Incluye inactivos	Interno	ADMIN
S24	R11	B4 nuevo vino	Crear producto	POST /api/v1/ admin/products	body {id slug, nombre, tipo, vina, volumenMl, gradoAlcohol, precio, imagen, recomendado, stock}	201 producto	409 id existente; imagen restringida a images/vinos/	Público al publicarse	ADMIN
S25	R11	B4 editar ficha	Modificar ficha	PUT /api/v1/ admin/products/{ id}	body ficha completa (sin stock)	200 producto	404. El stock tiene operación propia auditada	Público	ADMIN
S26	R11	B4 ajustar stock	Ajustar stock	PATCH /api/v1/ admin/products/{ id}/stock	body {delta ≠0, motivo enum}	200 producto	PATCH: cambio parcial; 409 si quedaría negativo; registra MovimientoStock	Interno	ADMIN

ID	Req.	Pantalla y acción	Propósito	Verbo y ruta	Entrada	Salida	Reglas y errores	Clasif.	Acceso
S27	R11	B4 retirar vino	Retirar del catálogo	DELETE /api/v1/admin/products/{id}	path id	200 {activo:false}	Eliminación lógica: los pedidos guardan copia de nombre y precio	Interno	ADMIN
S28	R11	B4 pedidos	Listar pedidos	GET /api/v1/admin/orders	query estado, pagina, tamano	200 {items[+clienteId, entrega]}	Datos de despacho sólo para ADMIN	Restringido	ADMIN
S29	R11	B4 despacho	Avanzar despacho	PATCH /api/v1/admin/orders/{id}/status	body {estado: EN_PREPARACION|DESPACHADO|ENTREGADO}	200 pedido con historial	409 INVALID_TRANSITION fuera de la secuencia	Restringido	ADMIN
S30	R08	B4 reservas	Listar reservas	GET /api/v1/admin/tastings/reservations	query fecha?	200 {items[+contacto, comentarios]}	Incluye datos de contacto y comentarios (sensibles)	Restringido + sensible	ADMIN, SOMMELIER
S31	R08	B4 asistencia	Gestionar reserva	PATCH /api/v1/admin/tastings/reservations/{id}/status	body {estado: ASISTIDA|NO_ASISTIO|CANCELADA}	200 reserva	409 si ya fue gestionada	Restringido	ADMIN, SOMMELIER
S32	R09	B4 mensajes	Listar mensajes	GET /api/v1/admin/contact	query estado, pagina, tamano	200 {items[...]}	—	Restringido	ADMIN
S33	R09	B4 atender	Marcar atendido	PATCH /api/v1/admin/contact/{id}	body {estado}	200	404	Restringido	ADMIN
S34	R13	B5 reportes	Panel de ventas	POST /api/v1/graphql · panelVentas	{panelVentas(desde, hasta)}	data.panelVentas { totalVentas, … , stockBajo}	Rango ≤ 1 año; sólo ADMIN; sin datos personales	Interno agregado	ADMIN

Códigos comunes a todas las rutas: 401 sin sesión válida, 403 rol sin permiso o CSRF, 404 ruta no publicada o recurso ajeno, 405 verbo no permitido, 413 cuerpo > 16 KB, 415 tipo de contenido no JSON, 429 límite de solicitudes, 503 dependencia no disponible, 504 tiempo de espera. Formato de error único: `{"error":{"code","message","requestId","details?"}}` sin trazas ni valores recibidos.

1.2 Ejemplos de contrato (datos ficticios)

ID	Solicitud	Respuesta
S01	GET /api/v1/products?tipo=tinto	{"items":[{"id":"kuyen-tinto","nombre":"Kuyen (blend tinto)","tipo":"tinto","vina":"Viña Antiyal","volumenMl":750,"gradoAlcohol":14.5,"precio":32000,"imagen":"images/vinos/kuyen-tinto.jpg","recomendado":true,"disponible":true}],"total":2,"pagina":1,"tamano":24}
S02	{"items":[{"productoId":"kuyen-tinto","cantidad":2}],"destino":{"comuna":"Providencia","region":"Metropolitana"}}	{"items":[{"productoId":"kuyen-tinto","precioUnitario":32000,"cantidad":2,"subtotal":64000,"disponible":true}],"noDisponibles":[],"subtotal":64000,"despacho":0,"total":64000}
S03	{"nombre":"Ana Pérez","email":"ana@correo.cl","password":"Clave2026x","fechaNacimiento":"1998-03-10","aceptaTerminos":true}	201 {"userId":"5c1e…","email":"ana@correo.cl","nombre":"Ana Pérez"}
S04	{"email":"ana@correo.cl","password":"Clave2026x"}	200 {"user":{"id":"5c1e…","email":"ana@correo.cl","rol":"CLIENTE"},"expiresIn":1800} + Set-Cookie: vu_session=…; HttpOnly; Secure; SameSite=Strict
S05	POST (sin cuerpo)	{"sesionCerrada":true}

ID	Solicitud	Respuesta
S06	GET	{"userId":"5c1e…","email":"ana@correo.cl","rol":"CLIENTE"}
S07	GET	{"nombre":"Ana Pérez","email":"ana@correo.cl","telefono":"+56 9 5555 1234","fechaNacimiento":"1998-03-10","direccion":null}
S08	{"nombre":"Ana Pérez","telefono":"+56 9 5555 1234","direccion":{"calle":"Los Leones","numero":"100","comuna":"Providencia","region":"Metropolitana"}}	200 (perfil actualizado)
S09	DELETE	{"eliminado":true} · 409 {"error":{"code":"ACTIVE_OPERATIONS",…}}
S10	Idempotency-Key: 3f9c… {"items":[{"productoId":"riesling-blanco","cantidad":2}],"entrega":{"nombreReceptor":"Ana Pérez","calle":"Los Leones","numero":"100","comuna":"Providencia","region":"Metropolitana","telefono":"+56955551234"}}	201 {"id":"a7d2…","numero":152,"estado":"PENDIENTE_PAGO","subtotal":55000,"despacho":0,"total":55000,"venceEn":"2026-11-09T15:15:00Z",…}
S11	{"query":"{ misPedidos(tamano:5){ total items{ numero estado total pago{ estado boleta{ folio } } } } }"}	{"data":{"misPedidos":{"total":1,"items":[{"numero":152,"estado":"PAGADO","total":55000,"pago":{"estado":"APROBADO","boleta":{"folio":31}}}]}}}
S12	{"query":"query($id:ID!){ resumenPedido(id:$id){ estado total } }","variables":{"id":"a7d2…"}}	{"data":{"resumenPedido":{"estado":"PAGADO","total":55000}}} · ajeno: {"data":{"resumenPedido":null}}
S13	POST /api/v1/orders/a7d2…/cancel	{"id":"a7d2…","estado":"CANCELADO","historial":[…]}
S14	{"pedidoId":"a7d2…"}	201 {"pagoId":"0eb0…","estado":"INICIADO","monto":55000,"redirectUrl":"https://localhost:8090/pay/13ee…"}
S15	POST /api/v1/payments/0eb0…/verify	{"estado":"APROBADO","monto":55000,"boleta":{"folio":31,"neto":46218,"iva":8782,"total":55000}}
S16	{"event_id":"9a1f…","token":"13ee…","status":"AUTHORIZED","amount":55000} + X-Provider-Signature: sha256=…	{"recibido":true,"duplicado":false}
S17	GET	{"items":[{"id":"iniciacion-cata","nombre":"Iniciación a la cata","precioPorPersona":15000,"duracionMin":60,…}]}
S18	GET ?fecha=2026-11-20	{"fecha":"2026-11-20","capacidad":12,"horarios":[{"hora":"10:00","cuposDisponibles":12},…]}
S19	{"paqueteId":"maridaje-gourmet","fecha":"2026-11-20","hora":"18:00","personas":4,"contacto":{"nombre":"Ana Pérez","email":"ana@correo.cl","telefono":"+56955551234"},"comentarios":"Uno es vegetariano"}	201 {"id":"b81c…","paquete":"Maridaje Gourmet","total":112000,"estado":"CONFIRMADA"}
S20	GET	{"items":[{"id":"b81c…","fecha":"2026-11-20","hora":"18:00","estado":"CONFIRMADA",…}]}
S21	POST …/b81c…/cancel	{"id":"b81c…","estado":"CANCELADA"}
S22	{"nombre":"Pedro","email":"pedro@correo.cl","asunto":"Pedido especial","mensaje":"¿Tienen cajas de regalo?"}	201 {"id":"c3…","estado":"PENDIENTE"}
S23	GET	{"items":[{"id":"kuyen-tinto",…,"stock":24,"activo":true}]}
S24	{"id":"carmenere-reserva","nombre":"Carmenere Reserva","tipo":"tinto","vina":"Viña X","volumenMl":750,"gradoAlcohol":13.5,"precio":15900,"imagen":"images/vinos/purafe-tinto.jpg","stock":12}	201 (producto)
S25	{"nombre":"Carmenere Reserva 2024",…,"precio":16900}	200 (producto)
S26	{"delta":12,"motivo":"INGRESO_BODEGA"}	200 {"id":"…","stock":36}
S27	DELETE	{"id":"carmenere-reserva","activo":false}
S28	GET ?estado=PAGADO	{"items":[{"numero":152,"clienteId":"5c1e…","entrega":{…}}],"total":3}
S29	{"estado":"EN_PREPARACION"}	200 (pedido) · 409 {"error":{"code":"INVALID_TRANSITION"}}
S30	GET ?fecha=2026-11-20	{"items":[{"paquete":"Maridaje Gourmet","contacto":{…},"comentarios":"Uno es vegetariano"}]}
S31	{"estado":"ASISTIDA"}	200 (reserva)

ID	Solicitud	Respuesta
S32	GET ?estado=PENDIENTE	{"items":[{"asunto":"Pedido especial","email":"pedro@correo.cl",. . . }]}
S33	{"estado":"ATENDIDO"}	{"id":"c3. . . ","estado":"ATENDIDO"}
S34	{"query":"{ panelVentas(desde:\"2026-11-01\",hasta:\"2026-11-30\"){ totalVentas pedidosPagados stockBajo{ nombre stock } } }"}	{"data":{"panelVentas":{"totalVentas":227600,"pedidosPagados":3,"stockBajo":[{"nombre" :"Viñedo Chadwick","stock":4}]}}}

Módulo 2 — Clases y cardinalidades del dominio

Se modelan las entidades del dominio con sus atributos, identificadores y sistema propietario, y se representan los servicios públicos con el estereotipo «service», cuyas operaciones se corresponden con los identificadores del módulo 1. Cada multiplicidad se respalda en una regla del negocio, y se distingue explícitamente entre el modelo interno persistido y los DTO expuestos al cliente.

El modelo se dividió en dos diagramas para que sea legible: (a) entidades internas con sus cardinalidades y el sistema dueño de cada una, y (b) servicios públicos «service» con sus operaciones (identificadas con los ID del módulo 1) y los DTO que exponen. Los archivos editables están en `docs/diagramas/` en formato draw.io (`uml_dominio.drawio`, `uml_servicios.drawio`) y Mermaid (`.mmd`), generados desde una misma definición (`generar_diagramas.py`) para que ambos formatos coincidan.

2.1 Leyenda de notación

Símbolo	**Significado en estos diagramas**
Rombo negro (composición)	La parte no existe sin el todo y se guarda dentro del mismo documento (p. ej. DetallePedido dentro de Pedido)
Rombo blanco (agregación)	La parte se agrupa en el todo pero tiene vida propia (ReservaDegustacion ocupa una SesionCata)
Flecha continua (asociación)	Referencia entre registros, normalmente por identificador y a veces entre sistemas distintos
Flecha discontinua (dependencia)	«use»/«create»: un servicio usa un contrato o DTO. No implica una relación entre registros de negocio
«entity · sistema»	Entidad interna y el microservicio que tiene autoridad sobre ella
+ / -	Atributo que puede salir en algún DTO / atributo interno que nunca sale al frontend
«copia»	Dato copiado al momento de la compra para conservar el historial

2.2 Diagrama de dominio

Figura no encontrada: `figuras/diagramas_uml_dominio.png`.
Suba la carpeta `figuras` al proyecto y vuelva a compilar.

Figura 1. UML de dominio (fuente editable: docs/diagramas/uml_dominio.drawio)

2.3 Relaciones, multiplicidades y reglas

Origen	**Destino**	**Mult. origen**	**Mult. destino**	**Regla del negocio que la origina**
Credencial	Sesion	1	0..*	Un usuario puede tener varias sesiones abiertas (dispositivos); cada sesión es de una credencial y se revoca sola
Credencial	PerfilCliente	1	0..1	Comparten userId. El personal (ADMIN, SOMMELIER) no tiene perfil de cliente; cada cliente tiene exactamente uno
PerfilCliente	Direccion	1	0..1	Dirección predeterminada opcional, sólo para prellenar el checkout
PerfilCliente	MensajeContacto	0..1	0..*	El formulario de contacto es anónimo; si el remitente tiene sesión se asocia
PerfilCliente	Pedido	1	0..*	Un cliente puede no haber comprado; cada pedido pertenece a un único cliente (regla de propiedad)
Pedido	DetallePedido	1	1..*	Un pedido confirmado contiene al menos un ítem; no se modela borrador vacío porque el carrito vive en el navegador

Origen	Destino	Mult. origen	Mult. destino	Regla del negocio que la origina
Pedido	Entrega	1	1	Todo pedido se despacha; se guarda copia de la dirección aunque luego cambie el perfil
Pedido	EventoPedido	1	1..*	El historial nace con PENDIENTE_PAGO y crece con cada transición
DetallePedido	Producto	0..*	1	Cada línea refiere a un producto; un producto puede no haberse vendido. Se copian nombre y precio
Pedido	ReservaStock	1	0..1	La reserva usa el mismo id del pedido; si la reserva falla el pedido no se crea
ReservaStock	Producto	0..*	1..*	Una reserva bloquea al menos un producto; un producto puede estar en muchas reservas
Producto	MovimientoStock	1	0..*	Cada ajuste manual de stock queda auditado
Pedido	Pago	1	0..*	Puede haber intentos rechazados y reintentos; a lo más uno INICIADO y uno APROBADO (índices únicos parciales)
Pago	Boleta	1	0..1	Sólo un pago aprobado emite boleta
Pago	NotificacionProveedor	1	0..*	El proveedor puede reenviar el mismo aviso; cada event_id se registra una vez
PaqueteDegustacion	ReservaDegustacion	1	0..*	Cada reserva elige un paquete
SesionCata	ReservaDegustacion	1	0..*	Varias reservas comparten la sala (capacidad 12 por fecha y hora)
PerfilCliente	ReservaDegustacion	1	0..*	Reservar requiere sesión de cliente mayor de edad

2.4 Servicios públicos y DTO

Figura no encontrada: `figuras/diagramas_uml_servicios.png`.
Suba la carpeta `figuras` al proyecto y vuelva a compilar.

Figura 2. Servicios «service» y DTO (fuente editable: docs/diagramas/uml_servicios.drawio)

Operación UML	ID	Operación UML	ID
AuthService.registrar	S03	OrderService.cotizar	S02
AuthService.iniciarSesion	S04	OrderService.crearPedido	S10
AuthService.cerrarSesion	S05	OrderService.anularPedido	S13
AuthService.consultarSesion	S06	OrderService.listarPedidos / cambiarEstado	S28 / S29
ClientService.consultarPerfil / modificarPerfil	S07 / S08	PaymentService.iniciarPago / verificarPago	S14 / S15
ClientService.eliminarCuenta	S09	PaymentService.recibirNotificacion	S16
ClientService.enviarContacto	S22	TastingService.consultarPaquetes / Disponibilidad	S17 / S18
ClientService.listarMensajes / atenderMensaje	S32 / S33	TastingService.reservar / misReservas / cancelar	S19 / S20 / S21
CatalogService.consultarCatalogo	S01	TastingService.listarReservas / gestionarReserva	S30 / S31
CatalogService.listarInventario … retirarProducto	S23-S27	AggregationService.misPedidos / resumenPedido / panelVentas	S11 / S12 / S34

2.5 Decisiones del equipo

- **¿Carrito y pedido son clases distintas?** Sí. El carrito es estado del cliente (en el frontend v1.0 vive en localStorage) y no se persiste en el servidor; aunque el frontend v1 guarda el precio en el carrito, el backend sólo acepta productoId y cantidad (S02, S10) y calcula los precios. El pedido nace en S10 con precios oficiales. Así un carrito abandonado no ocupa stock ni datos personales.
- **¿Qué pasa con el historial si un producto deja de estar disponible?** Retirar un vino (S27) es lógico (activo=false). DetallePedido guarda copia del nombre, tipo y precio, por lo que los

pedidos y boletas antiguos no cambian. GraphQL muestra además el estado actual del producto (campo `producto`, que puede ser null).

- **Modelo interno vs. DTO:** el stock exacto, el hash de contraseña, el token del proveedor, la clave de idempotencia y los datos de auditoría nunca salen. El DTO público de producto sólo dice `disponible`. `clienteId` sólo se entrega a ADMIN (y en GraphQL está protegido a nivel de campo).
- **Pedido–Pago 1 a 0..*:** se modelan los reintentos tras un rechazo; la unicidad del pago aprobado la garantiza un índice único parcial en la base de payments.

Módulo 3 — Sistemas e integraciones

Se distribuyen las responsabilidades entre los sistemas privados, se define la autoridad sobre cada dato y se especifican los contratos de integración entre componentes, incluyendo la identidad requerida en cada tramo y el comportamiento ante fallos de comunicación, respuestas tardías y notificaciones duplicadas.

Los cuatro sistemas mínimos (inventory, orders, clients, payments) son microservicios privados con base de datos propia. Se agregaron tres sistemas por necesidades reales del caso y por separación de credenciales: **auth** (credenciales y sesiones, para que ningún servicio de negocio vea hashes ni llaves de firma), **tastings** (degustaciones: ciclo de vida por cupos de sala, distinto al de un pedido con despacho) y **analytics** (reportes separados de las transacciones). No se dividió por tabla ni por pantalla: por ejemplo, despacho y boletas no son servicios propios porque comparten el ciclo de vida del pedido y del pago respectivamente.

3.1 Matriz de sistemas

Sistema	Responsabilidad	Datos que administra (autoridad)	Servicios	Dependencias	Contrato de integración	Credencial / identidad	Respuesta ante fallos
inventory	Catálogo, precios oficiales, stock y reservas de stock	products, reservations, stock_movements (vu_inventory)	S01, S23-S27; gRPC Quote/Reserve/Commit/Release	MongoDB, Vault	REST interno + gRPC (inventory.proto)	Token por tramo (gateway, graphql); mTLS CN=orders para gRPC	Reserva atómica por producto con compensación; idempotencia por reservation_id
orders	Cotización, pedidos, ciclo de vida y despacho	orders, counters, outbox (vu_orders)	S02, S10, S13, S28, S29	inventory (gRPC), analytics (eventos)	REST interno; payment-result desde payments	Tokens gateway/graphql/payments/clients; cert orders	Sin reserva confirmada no hay pedido; liberación idempotente; barrido de vencidos; reintentos de commit/release
clients	Perfil y mensajes de contacto	profiles, contact_messages (vu_clients)	S07-S09, S22, S32, S33	auth (baja), orders y tastings (operaciones activas)	REST interno	Tokens gateway, auth	Baja rechazada (503) si no puede verificar pedidos/reservas
payments	Inicio de pago, verificación con el proveedor, notificaciones y boletas	payments, webhook_events, counters (vu_payments)	S14, S15, S16	orders (monto y resultado), proveedor externo	REST interno + API del proveedor + webhook HMAC	Tokens gateway/graphql; API key del comercio y secreto de webhook en Vault	Notificación a orders con reintento; pago tardío → REQUIERE_REVERSA; doble aprobación bloqueada
auth	Credenciales, sesiones e introspección	credentials, sessions (vu_auth)	S03-S06	clients (crear perfil)	REST interno / internal/introspect	Llave JWT sólo en auth; tokens gateway, clients	Registro con compensación; Vault caído → usa llave en caché
tastings	Paquetes, cupos y reservas de degustación	packages, sessions, reservations, outbox (vu_tastings)	S17-S21, S30, S31	analytics (eventos)	REST interno	Tokens gateway, clients	Cupo con actualización condicional; purga de comentarios

Sistema	**Responsabilidad**	**Datos que administra (autoridad)**	**Servicios**	**Dependencias**	**Contrato de integración**	**Credencial / identidad**	**Respuesta ante fallos**
analytics	Reportes y métricas sin datos personales	events (vu_analytics)	S34 (vía GraphQL)	Eventos de orders y tastings	POST /internal/ events (idempotente por eventId)	Tokens orders, tastings, graphql	Outbox reintenta; duplicados descartados
GraphQL Gateway	Agregación de lectura	No tiene base de datos	S11, S12, S34	orders, inventory, payments, analytics	Schema GraphQL	Token gateway→graphql; tokens graphql→cada sistema	Errores parciales por campo; plazo 3 s por llamada
Proveedor (externo)	Cobro con tarjeta	Transacciones del proveedor	—	—	API REST + página de pago + webhook firmado	Api-Key-Id/Secret del comercio	Consulta servidor a servidor (verify) como respaldo del webhook

3.2 Mapa de componentes y despliegue

Figura no encontrada: `figuras/diagramas_mapa_sistemas_despliegue.png`.
Suba la carpeta `figuras` al proyecto y vuelva a compilar.

Figura 3. Mapa de sistemas y despliegue (fuente: docs/diagramas/mapa_sistemas_despliegue.mmd y docker-compose.yml)

Componente (Docker Compose)	Redes	Puerto publicado
gateway	edge, backend, secrets	8443 (HTTPS)
sandbox (proveedor simulado)	edge, egress, data, secrets	8090
auth, clients, inventory, orders, tastings, analytics	backend, data, secrets	ninguno
payments	backend, data, secrets, egress	ninguno
graphql	backend, secrets	ninguno
mongo	data	ninguno
vault, vault-init	secrets (+ data para vault-init)	ninguno

3.3 Secuencia representativa: compra y pago

Figura no encontrada: `figuras/diagramas_secuencia_compra_pago.png`.
Suba la carpeta `figuras` al proyecto y vuelva a compilar.

Figura 4. Secuencia de compra, pago y confirmación

3.4 Intercambio entre sistemas

Intercambio	Operación / evento	Datos mínimos	Autenticación	Errores y consistencia
orders → inventory	gRPC QuoteItems / ReserveStock / CommitReservation / ReleaseReservation	productoId, cantidad, reservation_id	mTLS (CN=orders)	Plazos 1-2 s; FAILED_PRECONDITION → 409; UNAVAILABLE → 503 y liberación compensatoria; operaciones idempotentes
payments → orders	GET /internal/orders/{id}; POST /internal/orders/{id}/payment-result	id, clienteId, estado, total · pagoId, resultado, monto	Token payments→orders	orders sólo pasa a PAGADO si estaba pendiente y el monto coincide; repetir es idempotente; 409 → reversa
proveedor → payments	Webhook	event_id, token, status, amount	HMAC-SHA256 con marca de tiempo (±5 min)	event_id único; la verificación (S15) consulta al proveedor si el webhook no llega
orders / tastings → analytics	order.paid, order.cancelled, order.delivered, booking.created	ids, productos, montos, región (sin datos personales)	Token por emisor; analytics valida que el emisor sea autoridad del evento	Outbox con reintentos cada 2 s; analytics descarta eventId repetido
auth → clients	POST /internal/clients	userId, nombre, email, teléfono, nacimiento	Token auth→clients	Si falla, auth borra la credencial (compensación)
clients → orders / tastings / auth	active-count; deactivate	userId	Tokens clients→*	Si no puede verificar, no da de baja (503)

Verificación de pagos y estado oficial

- La autoridad del estado del **pago** es payments; la del estado del **pedido** es orders; la del **stock** es inventory. Ningún sistema escribe datos de otro: colaboran por contratos.

- El aviso del proveedor se acepta sólo con firma HMAC válida y reciente; su `event_id` se inserta en una colección con clave única antes de aplicar efectos, de modo que un reenvío no emite dos boletas ni descuenta stock dos veces (prueba F03).
- El frontend nunca decide que un pedido está pagado: tras volver del proveedor sólo pide `verify`, y payments consulta al proveedor servidor a servidor (prueba V6).
- Si el pedido venció (15 min) antes de que llegue la aprobación, el pago se marca REQUIERE_REVERSA para devolución y el stock no vuelve a descontarse (prueba F14).

Ciclo de vida del pedido

Figura no encontrada: `figuras/diagramas_estados_pedido.png`.
Suba la carpeta `figuras` al proyecto y vuelva a compilar.

Figura 5. Transiciones permitidas del pedido (orders las valida con actualizaciones condicionales)

Módulo 4 — Stakeholders y permisos HTTP

Se registran las partes interesadas del caso, separando stakeholders, roles de usuario e identidades técnicas, y se define la matriz de permisos por servicio, ruta y verbo, con sus condiciones de alcance (propiedad del recurso, estado del pedido, firma del proveedor) y el componente responsable de verificarlas.

4.1 Registro de stakeholders

Stakeholder	Interés	Rol asociado	Uso de la información
Visitante	Conocer vinos, degustaciones y la tienda	anónimo	Catálogo, paquetes, cupos, cotización y contacto
Cliente registrado (mayor de 18)	Comprar, pagar, seguir pedidos y reservar catas	CLIENTE	Sus propios datos, pedidos, pagos, boletas y reservas
Administrador de la tienda	Catálogo, stock, despacho, mensajes y ventas	ADMIN	Operación completa del panel interno
Sommelier	Preparar y registrar las degustaciones	SOMMELIER	Sólo reservas (incluye comentarios sensibles)
Dueños / gerencia de Viña Urbana	Resultados del negocio	sin permisos operativos	Reciben reportes del panel de ventas exportados por el ADMIN
Productores artesanales (viñas)	Ventas de sus etiquetas	sin permisos operativos	Reciben del ADMIN el reporte de "más vendidos" (sin datos de clientes)
Empresa de despacho (courier)	Entregar pedidos	sin permisos operativos	El ADMIN le entrega la guía con nombre del receptor, dirección y teléfono
SII (boleta electrónica)	Cumplimiento tributario	sin permisos operativos	Recibiría las boletas; integración externa fuera de alcance (ver módulo 10)
Proveedor de pagos	Cobrar y notificar	identidad técnica	Webhook firmado S16; API del comercio
Componentes internos	Operar el sistema	identidades técnicas: gateway, graphql, auth, clients, orders, payments, tastings	Credencial por tramo en Vault (módulo 6)

Identidad y autorización son pasos distintos: el gateway **identifica** (introspección de la sesión en Auth) y aplica la primera autorización por ruta y rol; cada servicio **vuelve a autorizar** (rol, propiedad del recurso, estado). Ocultar un botón en el frontend no concede ni quita permisos.

4.2 Matriz de permisos por servicio, ruta y verbo

Rol o identidad	ID de servicio	Ruta	Verbo	Decisión	Alcance y condición	Dónde se verifica
Visitante	S01, S17, S18	/products, /tastings/packages, /tastings/availability	GET	Permitido	Sólo campos públicos	Gateway (ruta anónima) + servicio (DTO público)

Rol o identidad	ID de servicio	Ruta	Verbo	Decisión	Alcance y condición	Dónde se verifica
Visitante	S02, S22	/orders/quote, /contact	POST	Permitido	Límite 120/min y 5/min por IP; precio lo calcula el servidor	Gateway (límite, CSRF) + servicio
Visitante	S03, S04	/auth/register, /auth/login	POST	Permitido	≥18 años; 5 fallos bloquean 15 min	Auth
Visitante	S05-S16, S19-S21, S23-S34	resto	todos	Denegado (401)	Requiere sesión	Gateway
Cliente	S07 / S08 / S09	/clients/me	GET / PUT / DELETE	Permitido	Sólo su propio perfil (id tomado de la sesión)	Gateway (rol) + clients (propiedad)
Cliente	S10	/orders	POST	Permitido	Pedido a su nombre; Idempotency-Key	Gateway + orders
Cliente	S13	/orders/{id}/cancel	POST	Permitido con condición	Sólo si es suyo y está PENDIENTE_PAGO; si es ajeno 404	orders
Cliente	S11, S12	/graphql misPedidos, resumenPedido	POST	Permitido con condición	Sólo pedidos propios; campo clienteId denegado	GraphQL (rol, campo) + orders (propiedad)
Cliente	S14, S15	/payments, /payments/{id}/verify	POST	Permitido con condición	Pedido propio y pendiente; sin monto del navegador	payments + orders
Cliente	S19 / S20 / S21	/tastings/reservations...	POST / GET / POST	Permitido con condición	Reservas propias; cancelar con ≥24 h	tastings
Cliente	S23-S33	/admin/*	GET/POST/ PUT/PATCH/ DELETE	Denegado (403)	Consultar catálogo no otorga administración	Gateway + servicio
Cliente	S34	/graphql panelVentas	POST	Denegado (FORBIDDEN)	—	GraphQL + analytics
Administrador	S23-S27	/admin/products...	GET/POST/ PUT/PATCH/ DELETE	Permitido	DELETE es retiro lógico; stock no negativo	Gateway + inventory
Administrador	S28, S29	/admin/orders...	GET, PATCH	Permitido con condición	Sólo transiciones válidas de despacho	Gateway + orders
Administrador	S30-S33, S34, S12	reservas, mensajes, panel, resumen	GET/PATCH/ POST	Permitido	Ve datos de contacto para operar	Gateway + servicio
Administrador	S07-S10, S13-S15, S19-S21	operaciones de cliente	todos	Denegado (403)	El personal no compra con su cuenta	Gateway
Sommelier	S30, S31	/admin/tastings/reservations...	GET, PATCH	Permitido	Necesita comentarios (alergias) para preparar la cata	Gateway + tastings
Sommelier	resto de /admin/ *	/admin/products, orders, contact	todos	Denegado (403)	—	Gateway + servicio
Proveedor de pagos	S16	/payments/webhook	POST	Permitido con firma	HMAC válido, ±5 min, event_id único	payments
Identidad técnica gateway	todas	rutas internas de cada servicio	según ruta	Permitido	Credencial del tramo + rutas declaradas por servicio	CallerVerifier en cada servicio
Identidad técnica graphql	lecturas internas	/internal/orders, /internal/products/ batch, /internal/payments, /internal/ sales-summary	GET	Permitido	Propaga usuario verificado; nunca escribe	Cada servicio

Rol o identidad	ID de servicio	Ruta	Verbo	Decisión	Alcance y condición	Dónde se verifica
Identidad técnica payments	orders	/internal/orders/{id}, /payment-result	GET, POST	Permitido	No recibe dirección del cliente	orders
Identidad técnica orders	inventory gRPC	4 métodos	RPC	Permitido	Sólo certificado CN=orders	inventory (gRPC)

4.3 Verbos por recurso

Recurso	GET	POST	PUT	PATCH	DELETE
/products	Permitido (anónimo)	No aplica	No aplica	No aplica	No aplica (405)
/admin/products[/{id}]	ADMIN	ADMIN (crear)	ADMIN (ficha)	ADMIN (stock)	ADMIN (retiro lógico)
/orders	No aplica (lectura vía GraphQL)	CLIENTE	No aplica	No aplica	No aplica: anular = POST /cancel, conserva historial
/admin/orders[/{id}/status]	ADMIN	No aplica	No aplica	ADMIN (transición)	Denegado: los pedidos no se borran
/clients/me	CLIENTE (propio)	No aplica	CLIENTE (propio)	No aplica	CLIENTE (anonimiza)
/payments[/{id}/verify]	No aplica	CLIENTE (propio) / proveedor (webhook)	No aplica	No aplica	No aplica
/tastings/reservations	CLIENTE (/me)	CLIENTE	No aplica	ADMIN, SOMMELIER (/admin/. . . /status)	No aplica: cancelar = POST /cancel
/contact	ADMIN (/admin/ contact)	Anónimo	No aplica	ADMIN	No aplica (retención automática 12 meses)

PATCH se usa para cambios parciales con semántica propia (ajustar stock por delta, cambiar sólo el estado). La matriz permite comprobar que un usuario no consulta pedidos de otro: las pruebas V4 lo verifican en ejecución (GraphQL devuelve null; anular o pagar un pedido ajeno devuelve 404).

Módulo 5 — Clases del API Gateway

Se justifica el número de gateways, se clasifican las rutas por tipo de acceso y se modelan las clases del gateway, sus políticas y su relación con el proveedor de secretos y con el GraphQL Gateway. Se precisa, además, la división de validaciones entre el gateway y los servicios internos (defensa en profundidad) y las fronteras de confianza del sistema.

5.1 Cuántos gateways y por qué

Se usa **un API Gateway externo** (único punto público) y **un GraphQL Gateway privado** detrás de él. Un segundo gateway externo no se justifica: los distintos tipos de ruta (anónimas, autenticadas, administrativas) se separan con políticas por ruta dentro del mismo gateway. El GraphQL Gateway es privado porque su función es agregar datos internos, no exponerse; el navegador lo alcanza sólo como la ruta `POST /api/v1/graphql`, autenticada por el gateway externo.

Clase de ruta	Ejemplos	Tratamiento
Externa anónima	S01, S02, S03, S04, S17, S18, S22	Sin sesión; límites por IP; CSRF en POST; sólo datos públicos o captura de datos propios
Externa autenticada	S05-S15, S19-S21, G01	Introspección de sesión; rol requerido; el servicio verifica propiedad. Una ruta pública de red puede transportar datos restringidos del propio usuario
Administrativa	S23-S34	Rol ADMIN o SOMMELIER en el gateway y otra vez en el servicio
De proveedor	S16	Sin sesión; la autenticación es la firma HMAC, validada en payments
Interna (no publicada)	/internal/*, gRPC :50051, /graphql del GraphQL Gateway	No existen en el gateway (404). Sólo alcanzables en la red backend con credencial de servicio

5.2 Diagrama de clases del gateway y sus políticas

Figura no encontrada: `figuras/diagramas_uml_gateway.png`.
Suba la carpeta `figuras` al proyecto y vuelva a compilar.

Figura 6. UML del API Gateway, políticas, proveedor de secretos y GraphQL Gateway (fuente: docs/diagramas/uml_gateway.drawio)

Clase	Implementación	Rol
ApiGateway	backend/services/gateway/app.py	Recibe la solicitud, aplica políticas en orden y hace el proxy controlado
RouteDefinition (1..* por gateway)	backend/services/gateway/routes.py (lista ROUTES)	Configuración declarativa: verbo, patrón externo, acceso, roles, destino, parámetros y cabeceras permitidas, límite, plazo, tamaño
ServiceTarget	TARGETS en app.py (URL https por servicio)	Destino interno; cada RouteDefinition apunta a exactamente uno
AuthenticationPolicy + AuthServiceClient	introspect() → Auth /internal/introspect	Valida firma, vencimiento y revocación del token; el rol vigente se lee de la base
AuthorizationPolicy	identity.role ∈ route.roles	Primera barrera por rol
RequestPolicy	CSRF, rate_limited(), tamaño, Content-Type, STRIPPED	Controles transversales y descarte de cabeceras de identidad
SecretProvider	vu_common/vault.py y readSecret() en GraphQL	Lee la credencial del tramo; caché 60 s; reintento forzado ante rotación
CallerVerifier	vu_common/internal.py require_caller()	En cada servicio: valida identidad del llamador y que esa ruta acepte a ese llamador
GraphQLGateway, Schema, Resolver, DataLoader, SubsystemAdapter	backend/gql-gateway/src	Agregación privada (módulo 8)

Clase	Implementación	Rol
InventoryGrpcClient	backend/services/orders/inventory_client.py	Cliente gRPC con mTLS y plazos

Relaciones estructurales: ApiGateway «1»–«1..*» RouteDefinition (composición: las rutas no existen fuera del gateway); RouteDefinition «0..*»–«1» ServiceTarget; GraphQLGateway «1»–«1» Schema «1»–«1..*» Resolver. Las flechas discontinuas son dependencias de uso (por ejemplo ApiGateway depende de SecretProvider para obtener la credencial del tramo); no representan flujos de red, que se muestran en el mapa del módulo 3.

5.3 Qué valida cada capa

Validación	API Gateway	Servicio interno
Ruta y verbo publicados	Sí (lista explícita, 404/405)	—
Sesión válida y no revocada	Sí (introspección)	—
Rol	Sí (por ruta)	Sí, otra vez (require_role)
Identidad del componente que llama	—	Sí (credencial del tramo; 401 si falta)
Propiedad del recurso	—	Sí (clienteId == usuario verificado)
Reglas de negocio y transiciones	—	Sí
Esquema del cuerpo	Tamaño y tipo	Sí (Pydantic, campos extra prohibidos donde aplica)
Límites de solicitudes, CSRF, cabeceras de seguridad	Sí	—
Tiempos de espera	Sí (8-12 s por ruta)	Sí hacia sus dependencias (gRPC 1-2 s, REST 3-4 s)

5.4 Fronteras de confianza

- **Internet → gateway:** todo lo que llega es no confiable. El gateway descarta `X-User-Id`, `X-User-Role`, `X-Caller-Id`, `X-Service-Token` y `X-Forwarded-*` enviados por el navegador y sólo reenvía parámetros y cabeceras declarados en la ruta (no hay reenvío genérico).
- **Gateway → servicios:** cada servicio confía en `X-User-*` sólo si el llamador presentó una credencial válida de gateway o graphql. Los demás llamadores internos no pueden propagar identidad de usuario.
- **Servicios → datos y secretos:** cada servicio tiene usuario de MongoDB sobre su base y un token de Vault con política propia. Vault y MongoDB no tienen puertos publicados.

5.5 Recorrido de una solicitud autorizada y una denegada

Figura no encontrada: `figuras/diagramas_secuencia_autorizada_denegada.png`. Suba la carpeta `figuras` al proyecto y vuelva a compilar.

Figura 7. El cliente A obtiene su pedido; el cliente B recibe null para el mismo id; una llamada directa a orders sin credencial recibe 401

La redirección del gateway no es genérica: restringe destinos (siete ServiceTarget), rutas (31 RouteDefinition REST + 1 de GraphQL) y verbos. Los logs del gateway registran método, ruta sin query string, estado, duración y request_id, nunca tokens ni cuerpos.

Módulo 6 — Rutas y secretos de integración

Se documenta la lista explícita de rutas publicadas con su transformación y destino interno, las operaciones GraphQL y los métodos gRPC, y el inventario simbólico de secretos e identidades con su emisor, consumidor, finalidad, ubicación, alcance y procedimiento de rotación y revocación. No se incluyen valores reales de credenciales.

6.1 Matriz de rutas REST

Todas las rutas pertenecen al único gateway externo. Transformación común: se elimina el prefijo `/api/v1`, se filtran parámetros y cabeceras, se agregan `X-Caller-Id: gateway`, la credencial del tramo, `X-Request-Id` y, si hay sesión, `X-User-Id`/`X-User-Role` verificados. Destinos internos: `https://<servicio>:8000`.

ID	Gateway	Verbo	Ruta externa	Acceso y rol	Sistema y ruta destino	Controles	Referencia de credencial
S01	externo	GET	/api/v1/products	Anónimo	inventory: GET /products	CSRF (POST), límite por IP, tamaño 16 KB	internal/inventory/ gateway
S02	externo	POST	/api/v1/orders/quote	Anónimo	orders: POST /orders/quote	CSRF (POST), límite por IP, tamaño 16 KB	internal/orders/ gateway
S03	externo	POST	/api/v1/auth/register	Anónimo (límite 10/min IP)	auth: POST /register	CSRF (POST), límite por IP, tamaño 16 KB	internal/auth/ gateway
S04	externo	POST	/api/v1/auth/login	Anónimo (límite 20/min IP)	auth: POST /login	CSRF (POST), límite por IP, tamaño 16 KB	internal/auth/ gateway
S05	externo	POST	/api/v1/auth/logout	Autenticado (todos)	auth: POST /internal/sessions/{jti}/ revoke	introspección, rol, límite por usuario, CSRF	internal/auth/ gateway
S06	externo	GET	/api/v1/auth/session	Autenticado (todos)	auth: GET /session	introspección, rol, límite por usuario, CSRF	internal/auth/ gateway
S07	externo	GET	/api/v1/clients/me	CLIENTE	clients: GET /clients/me	introspección, rol, límite por usuario, CSRF	internal/clients/ gateway
S08	externo	PUT	/api/v1/clients/me	CLIENTE	clients: PUT /clients/me	introspección, rol, límite por usuario, CSRF	internal/clients/ gateway
S09	externo	DELETE	/api/v1/clients/me	CLIENTE	clients: DELETE /clients/me	introspección, rol, límite por usuario, CSRF	internal/clients/ gateway
S10	externo	POST	/api/v1/orders	CLIENTE	orders: POST /orders	introspección, rol, límite por usuario, CSRF	internal/orders/ gateway
S13	externo	POST	/api/v1/orders/{id}/cancel	CLIENTE	orders: POST /orders/{id}/cancel	introspección, rol, límite por usuario, CSRF	internal/orders/ gateway
S14	externo	POST	/api/v1/payments	CLIENTE	payments: POST /payments	introspección, rol, límite por usuario, CSRF	internal/payments/ gateway
S15	externo	POST	/api/v1/payments/{id}/verify	CLIENTE	payments: POST /payments/{id}/ verify	introspección, rol, límite por usuario, CSRF	internal/payments/ gateway
S16	externo	POST	/api/v1/payments/webhook	Identidad del proveedor (HMAC)	payments: POST /payments/ webhook	firma HMAC (en payments), límite 60/min	internal/payments/ gateway

ID	Gateway	Verbo	Ruta externa	Acceso y rol	Sistema y ruta destino	Controles	Referencia de credencial
S17	externo	GET	/api/v1/tastings/packages	Anónimo	tastings: GET /tastings/packages	CSRF (POST), límite por IP, tamaño 16 KB	internal/tastings/ gateway
S18	externo	GET	/api/v1/tastings/availability	Anónimo	tastings: GET /tastings/availability	CSRF (POST), límite por IP, tamaño 16 KB	internal/tastings/ gateway
S19	externo	POST	/api/v1/tastings/reservations	CLIENTE	tastings: POST /tastings/reservations	introspección, rol, límite por usuario, CSRF	internal/tastings/ gateway
S20	externo	GET	/api/v1/tastings/reservations/me	CLIENTE	tastings: GET /tastings/reservations/ me	introspección, rol, límite por usuario, CSRF	internal/tastings/ gateway
S21	externo	POST	/api/v1/tastings/reservations/{id}/ cancel	CLIENTE	tastings: POST /tastings/ reservations/{id}/cancel	introspección, rol, límite por usuario, CSRF	internal/tastings/ gateway
S22	externo	POST	/api/v1/contact	Anónimo	clients: POST /contact	CSRF (POST), límite por IP, tamaño 16 KB	internal/clients/ gateway
S23	externo	GET	/api/v1/admin/products	ADMIN	inventory: GET /admin/products	introspección, rol, límite por usuario, CSRF	internal/inventory/ gateway
S24	externo	POST	/api/v1/admin/products	ADMIN	inventory: POST /admin/products	introspección, rol, límite por usuario, CSRF	internal/inventory/ gateway
S25	externo	PUT	/api/v1/admin/products/{id}	ADMIN	inventory: PUT /admin/products/{ id}	introspección, rol, límite por usuario, CSRF	internal/inventory/ gateway
S26	externo	PATCH	/api/v1/admin/products/{id}/stock	ADMIN	inventory: PATCH /admin/products/ {id}/stock	introspección, rol, límite por usuario, CSRF	internal/inventory/ gateway
S27	externo	DELETE	/api/v1/admin/products/{id}	ADMIN	inventory: DELETE /admin/ products/{id}	introspección, rol, límite por usuario, CSRF	internal/inventory/ gateway
S28	externo	GET	/api/v1/admin/orders	ADMIN	orders: GET /admin/orders	introspección, rol, límite por usuario, CSRF	internal/orders/ gateway
S29	externo	PATCH	/api/v1/admin/orders/{id}/status	ADMIN	orders: PATCH /admin/orders/{id}/ status	introspección, rol, límite por usuario, CSRF	internal/orders/ gateway
S30	externo	GET	/api/v1/admin/tastings/reservations	ADMIN, SOMMELIER	tastings: GET /admin/tastings/ reservations	introspección, rol, límite por usuario, CSRF	internal/tastings/ gateway
S31	externo	PATCH	/api/v1/admin/tastings/ reservations/{id}/status	ADMIN, SOMMELIER	tastings: PATCH /admin/tastings/ reservations/{id}/status	introspección, rol, límite por usuario, CSRF	internal/tastings/ gateway
S32	externo	GET	/api/v1/admin/contact	ADMIN	clients: GET /admin/contact	introspección, rol, límite por usuario, CSRF	internal/clients/ gateway
S33	externo	PATCH	/api/v1/admin/contact/{id}	ADMIN	clients: PATCH /admin/contact/{id}	introspección, rol, límite por usuario, CSRF	internal/clients/ gateway
G01	externo	POST	/api/v1/graphql	Autenticado (todos)	graphql: POST /graphql	introspección, 60/min/ usuario, 8 KB, CSRF	internal/graphql/ gateway

6.2 Operaciones GraphQL

Operación	Tipo	Campos autorizados	Resolver	Subsistemas consultados	Regla
misPedidos(pagina, tamano)	Query	Todos los de ResumenPedido salvo clienteId	Query.misPedidos → orders; ItemPedido.producto → DataLoader inventory; ResumenPedido.pago → DataLoader payments	orders, inventory, payments	Rol CLIENTE; tamano 1-10; sólo propios
resumenPedido(id)	Query	Igual; clienteId sólo ADMIN	Query.resumenPedido + los mismos resolvers de campo	orders, inventory, payments	Dueño o ADMIN (orders responde 404 → null)
panelVentas(desde, hasta)	Query	Todos	Query.panelVentas → analytics; PanelVentas. stockBajo → inventory	analytics, inventory	Rol ADMIN (verificado en GraphQL y en analytics)
Mutations	—	—	—	—	No se exponen: crear, anular y pagar quedan en los servicios de negocio (REST)

`POST /graphql` no autoriza todas sus operaciones: cada resolver exige su rol, los campos sensibles tienen su propia regla y los servicios aplican propiedad. Límites: profundidad 4, costo 2.500, sin introspección, sin lotes de operaciones, cuerpo 8 KB.

6.3 Métodos gRPC

Servicio y método	Entrada	Salida	Modalidad	Identidad permitida	Plazo máximo
InventoryReservations/QuoteItems	QuoteRequest{items[product_id, quantity]}	QuoteResponse{items[unit_price, active, available…]}	Unary	CN=orders (mTLS)	1 s
InventoryReservations/ReserveStock	ReserveRequest{reservation_id, items}	ReservationResult{status HELD}	Unary, idempotente	CN=orders	2 s
InventoryReservations/CommitReservation	ReservationRef{reservation_id}	ReservationResult{COMMITTED}	Unary, idempotente	CN=orders	2 s
InventoryReservations/ReleaseReservation	ReservationRef{reservation_id, reason}	ReservationResult{RELEASED}	Unary, idempotente	CN=orders	2 s

6.4 Credencial del usuario vs. credencial entre sistemas

- **Usuario:** cookie `vu_session` (JWT HS256 de 30 min con `jti`) que sólo viaja navegador ↔ gateway. Ningún servicio interno recibe el token del usuario.
- **Entre sistemas:** credencial aleatoria distinta por tramo `secret/internal/<destino>/<llamador>`. El gateway convierte la sesión en identidad verificada (`X-User-Id`, `X-User-Role`) y el destino sólo la acepta si la credencial del tramo es válida; así la comprobación de propiedad usa una identidad confiable.
- **gRPC:** identidad por certificado de cliente (mTLS); no requiere secreto estático. Si TLS se desactivara en desarrollo, se usa el token `internal/inventory/orders`.

6.5 Inventario simbólico de secretos e identidades

Referencia	Tipo	Emisor	Consumidor	Finalidad	Ubicación segura	Alcance	Rotación y revocación
auth/jwt	Llave HMAC (signing_key, kid, previous_key)	vault-init	auth	Firmar y validar sesiones	Vault KV	Sólo política vu-auth	`--rotate auth/jwt`; la anterior se acepta hasta que vencen los tokens (30 min)
internal/<destino>/<llamador> (19 tramos)	Token aleatorio 256 bits	vault-init	llamador y destino	Identidad servicio a servicio	Vault KV	Lectura sólo para esos dos servicios	`--rotate internal/x/y`; caché 60 s y relectura ante 401 (prueba R04)
db/<servicio> (8)	Usuario/contraseña MongoDB	vault-init (createUser)	el servicio dueño	Acceso a su base	Vault KV	readWrite sólo en vu_<servicio>	Re-ejecutar vault-init actualiza la contraseña (updateUser)
payments/provider	Api-Key-Id, Api-Key-Secret, webhook_secret	Proveedor (simulado: vault-init)	payments	API del comercio y firma de avisos	Vault KV	Sólo vu-payments	Se reemplaza en el portal del proveedor y en Vault
payments/transbank	Código de comercio y API key	Transbank	payments	Webpay Plus (opcional)	Vault KV (desde .env, nunca en código)	Sólo vu-payments	Según portal Transbank
sandbox/merchant	Copia del lado del proveedor	vault-init	sandbox	Validar al comercio y firmar avisos	Vault KV	Sólo vu-sandbox	Igual que payments/provider
Token de Vault por servicio (10)	Token con política vu-<svc>, TTL 720 h	vault-init	cada servicio	Leer sus secretos	runtime/tokens/<svc>/ montado sólo en ese contenedor	Mínimo privilegio (prueba de política)	Cada arranque emite uno nuevo y revoca el anterior
Certificados TLS (CA interna + 12)	X.509 ECDSA P-256, 90 días	setup.py	cada componente	TLS en todos los tramos; mTLS gRPC	runtime/certs/<svc>/ (la llave de la CA no se monta)	Un certificado por componente	Regenerar con setup.py –force; revocar = reemitir CA
MONGO_ROOT_PASSWORD, VAULT_ROOT_TOKEN	Credenciales de arranque	setup.py (aleatorio)	mongo, vault, vault-init	Inicialización	.env (gitignore)	Sólo contenedores de infraestructura	Regenerar .env y recrear volúmenes

Decisiones de seguridad

- No hay un secreto compartido por todos: 19 credenciales de tramo, 8 usuarios de base y 10 tokens de Vault, cada uno con un consumidor y una finalidad. La credencial de payments sólo sirve para payments.
- La credencial de base de datos pertenece al servicio que accede a ella; ni el navegador ni el gateway la conocen.
- Ningún secreto en rutas, query strings, repositorio, capturas ni logs (prueba `test_logs_sin_secretos_ni_datos_personales`). `.env.example` sólo trae referencias.
- **Si una clave se filtra:** se rota la referencia afectada con `vault_init.py --rotate`; los servicios toman el nuevo valor sin reinicio y el anterior deja de ser aceptado al vencer la caché (≤60 s; 10 s en el perfil de pruebas). Para la llave de sesiones se rota `auth/jwt` y se revocan sesiones.
- **Si Vault no responde:** los servicios siguen con el último secreto válido en caché (prueba R03); un servicio que nunca obtuvo su secreto falla cerrado

(503) en vez de operar sin credencial.

Módulo 7 — Datos sensibles y secretos del cliente

Se clasifican los datos y credenciales que maneja el sistema, se determina si la solución debe custodiar secretos asociados al cliente, se definen las medidas de almacenamiento, transporte, retención y exclusión de logs, y se presentan los resultados de los casos de validación de seguridad ejecutados contra el backend.

7.1 Matriz de datos y credenciales

Dato o credencial	Finalidad	Clasificación	Custodio	Roles autorizados	Representación en el frontend	Protección	Retención
Ficha de vino, precio, paquetes	Vender	Público	inventory, tastings	Todos	Catálogo	Integridad: sólo ADMIN modifica	Mientras exista; retiro lógico
Stock exacto	Operación	Interno	inventory	ADMIN	Sólo "disponible / agotado"	No sale en DTO público	Permanente
Nombre, email, teléfono	Cuenta y contacto	Personal	clients (email de acceso en auth)	Titular, ADMIN (vía pedidos)	Perfil propio	TLS, propiedad, no en logs (email enmascarado)	Hasta la baja: se anonimiza
Fecha de nacimiento	Cumplir venta a >18 (R01)	Personal	clients	Titular	Perfil	Se valida en servidor	Hasta la baja
Dirección de entrega	Despacho	Personal	orders (copia por pedido)	Titular, ADMIN	Checkout y mis pedidos	No viaja a payments ni a analytics	Con el pedido (respaldo tributario 6 años)
Historial de pedidos	Seguimiento	Personal	orders, payments	Titular, ADMIN	Mis pedidos (GraphQL)	Propiedad en orders; GraphQL restringe campos	Respaldo tributario
Comentarios de la reserva (alergias)	Preparar la cata	**Sensible (salud)**	tastings	Titular, SOMMELIER, ADMIN	Formulario de reserva	Sólo staff y titular; minimización	Se borran 30 días después de la cata
Mensajes de contacto	Atención	Personal	clients	ADMIN	Formulario	Límite 5/min	12 meses (índice TTL)
Contraseña	Autenticación	Credencial	auth	Ninguno (sólo verificación)	Campo password	**Argon2id con sal** (no cifrado reversible); nunca se devuelve	Se borra el hash en la baja
Sesión (JWT con jti)	Autenticación	Credencial	auth (sesiones) / navegador (cookie)	Titular	Cookie HttpOnly invisible para JS	Secure, SameSite=Strict, 30 min, revocable	Índice TTL elimina sesiones vencidas
Datos de tarjeta	Pagar	Credencial financiera	**Proveedor**	Ningún rol del comercio	Página del proveedor	Nunca ingresan al sistema	No aplica
Token de transacción del proveedor	Consultar el pago	Interno	payments	payments	No se muestra	No sale en DTO	Con el pago
Llaves del comercio y webhook	Integración de pagos	Secreto de integración	Vault	payments	Nunca	Política de Vault	Rotación
Eventos analíticos	Reportes	Agregado sin identificación	analytics	ADMIN vía GraphQL	Panel	Sin clienteId, email, dirección; filtro adicional al ingerir	Permanente

7.2 Decisión sobre secretos del cliente

Decisión: NO. Viña Urbana no custodia secretos asociados al cliente. Los datos de tarjeta se ingresan sólo en la página del proveedor; el sistema guarda el resultado y un token de transacción que no permite cobrar. La contraseña no es un secreto custodiado sino una credencial verificada contra un hash Argon2id irreversible. La cookie de sesión es una credencial temporal y revocable, no un secreto de larga vida. Lo que sí se protege con rigor son datos personales (dirección, teléfono, fecha de nacimiento) y un dato sensible (comentarios con posibles alergias), que no son claves.

Si en el futuro se ofrecieran pagos recurrentes, se usaría la tokenización del proveedor (token que sólo sirve para ese comercio) y no se guardarían números de tarjeta.

7.3 Recorrido de un dato restringido: la dirección de entrega

- **Captura:** el cliente la envía en el cuerpo de S10 (checkout, brecha B1) sobre HTTPS.
- **Validación:** orders valida formato, región y teléfono; se guarda copia dentro del pedido (vu_orders) con acceso sólo del servicio dueño.
- **Uso:** el titular la consulta en sus pedidos (S11, campos de entrega); el ADMIN la ve para despachar (S28) y la entrega al courier. payments recibe sólo id, cliente, estado y total; analytics sólo la región.
- **Exclusión de logs:** los logs no registran cuerpos ni query strings.
- **Fin:** si el cliente elimina su cuenta (S09), el perfil se anonimiza y la credencial se desactiva; la copia en pedidos pagados se conserva como respaldo tributario (justificación legal) y deja de estar asociada a una cuenta utilizable.

Recorrido análogo para los comentarios de reserva: capturados desde el formulario de P05 (S19) → guardados en tastings → visibles para sommelier/ADMIN (S30) → borrados automáticamente 30 días después de la fecha (hilo de purga).

7.4 Sesiones y tokens

- JWT HS256 firmado por Auth con llave en Vault; 30 minutos; claims sub, role, jti, iss, aud. El gateway no confía en el token: lo introspecta en cada solicitud, lo que permite revocación inmediata (logout, baja de cuenta) y que un cambio de rol aplique de inmediato.
- Cookie HttpOnly + Secure + SameSite=Strict + Path=/api: JavaScript no puede leerla (mitiga robo por XSS) y no viaja en solicitudes de otros sitios. Además se exige `X-Requested-With` en operaciones que cambian estado (CSRF).
- Un navegador no puede guardar confidencialmente una clave compartida incluida en su código; por eso el contrato no exige ninguna clave al cliente (el proveedor de pagos se usa por redirección) y el frontend, al integrarse, no necesitará secretos.

7.5 Casos de validación ejecutados

Caso	Entrada	Identidad	Resultado esperado	Componente que verifica	Resultado obtenido (prueba)
V1 Acceso anónimo al catálogo	GET /products, /tastings/ packages	Ninguna	200 con campos públicos	Gateway + inventory	Aprobado (test_V1, test_S01_ catalogo_solo_campos_publicos)
V2 Operación protegida sin sesión	POST /orders, GET /clients/ me, POST /graphql…	Ninguna / token falso / token revocado	401 UNAUTHENTICATED	Gateway (introspección)	Aprobado: 7 rutas + token falso + logout (9 casos)
V3 Administración sin permiso	GET/POST/DELETE / admin/products, GET / admin/orders	CLIENTE; SOMMELIER; cabeceras X-User-Role falsas	403	Gateway + servicio	Aprobado (8 casos)
V4 Pedido ajeno	resumenPedido(id de A), cancel, payments	Cliente B	null / 404 sin datos de A	orders (propiedad)	Aprobado (5 casos)
V5 Acceso directo al backend	https://inventory:8000/ products, /internal/*, / graphql, gRPC	Sin credencial / credencial adivinada / cert de otro componente	401 / PERMISSION_ DENIED / handshake rechazado	CallerVerifier, mTLS	Aprobado (10 casos)
V6 Pago alterado desde el frontend	total/monto en el cuerpo; webhook falso; verify sin aprobación	Cliente	400; 401; sigue INICIADO/ PENDIENTE_PAGO	orders, payments	Aprobado (3 casos)

Respuestas al control: la entrega no incluye código de frontend y el frontend v1.0 no contiene claves; no hay secretos en el código, capturas ni ejemplos (los ejemplos usan datos ficticios y referencias simbólicas); los DTO devuelven lo mínimo necesario; aunque la pantalla sea accesible desde Internet, los datos del cliente requieren sesión, rol y propiedad verificados en el servidor.

Módulo 8 — Separación privada y agregaciones GraphQL

Se clasifican los servicios privados según su naturaleza (borde, transaccional, agregación y analítica) y se presenta el GraphQL Gateway privado: schema, resolvers, matriz de autorización por operación y campo, límites de profundidad y costo, estrategia de batching y tratamiento de errores parciales, junto con la evidencia de su ejecución a través del API Gateway.

8.1 Mapa de clasificación de servicios

Servicio	Naturaleza	Responsabilidad	Datos propios	Contrato	Dependencias	Justificación de la separación
API Gateway	Borde (edge)	Entrada única y políticas	Ninguno	REST público / api/v1	Auth, todos	Única pieza expuesta; concentra controles transversales
auth	Transaccional (identidad)	Credenciales y sesiones	vu_auth	REST interno	clients	Aísla hashes y llave de firma; cambia por motivos de seguridad, no de negocio
clients	Transaccional	Perfil y contacto	vu_clients	REST interno	auth, orders, tastings	Datos personales con reglas de acceso propias
inventory	Transaccional + alta demanda	Catálogo y stock	vu_inventory	REST + gRPC	—	Lectura más frecuente del sistema (catálogo); concurrencia sobre el stock
orders	Transaccional	Pedidos y despacho	vu_orders	REST interno	inventory (gRPC), analytics	Ciclo de vida del pedido; alta cohesión con despacho
payments	Transaccional + integración externa	Pagos y boletas	vu_payments	REST + webhook	orders, proveedor	Única pieza con salida al proveedor y llaves del comercio (red egress)
tastings	Transaccional	Degustaciones	vu_tastings	REST interno	analytics	Ciclo de vida por cupos, distinto al de pedidos
analytics	Analítica	Métricas	vu_analytics	Eventos + REST interno	orders, tastings (eventos)	Separa consultas de reporte de las transacciones; sin datos personales
GraphQL Gateway	Agregación	Componer vistas de lectura	Ninguno	GraphQL privado	orders, inventory, payments, analytics	Evita que el frontend llame a varios sistemas y que se expongan; no escribe

Límites por cohesión y propiedad de datos: cada servicio modifica sólo sus colecciones (lo garantiza también MongoDB: el usuario de inventory no puede leer vu_auth, prueba `test_servicio_no_lee_base_de_otro_servicio`). La agregación nunca consulta bases de otros servicios: llama a sus APIs internas.

8.2 Schema

```
"""
GraphQL Gateway privado de Viña Urbana.
Sólo es alcanzable a través del API Gateway (POST /api/v1/graphql).
Cada operación y algunos campos tienen su propia regla de autorización.
Sólo hay consultas: las reglas transaccionales (crear, anular, pagar) quedan
en los servicios de negocio vía REST.
"""
type Query {
  "S11 - Rol CLIENTE. Pedidos propios con productos (inventory) y pago/boleta (payments)."
  misPedidos(pagina: Int = 1, tamano: Int = 5): PaginaPedidos!
```

```
  "S12 - Dueño del pedido o ADMIN. Combina orders + inventory + payments."
  resumenPedido(id: ID!): ResumenPedido
  "S34 - Rol ADMIN. Combina analytics (ventas) + inventory (stock bajo)."
  panelVentas(desde: String!, hasta: String!): PanelVentas!
}

type PaginaPedidos {
  items: [ResumenPedido!]!
  total: Int!
  pagina: Int!
  tamano: Int!
}

enum EstadoPedido {
  PENDIENTE_PAGO
  PAGADO
  EN_PREPARACION
  DESPACHADO
  ENTREGADO
  CANCELADO
}

type ResumenPedido {
  id: ID!
  numero: Int!
  estado: EstadoPedido!
  creadoEn: String!
  venceEn: String
  subtotal: Int!
  despacho: Int!
  total: Int!
  items: [ItemPedido!]!
  entrega: Entrega!
  historial: [EventoPedido!]!
  "Desde payments. Null si payments no responde (error parcial)."
  pago: Pago
  "Sólo ADMIN."
  clienteId: ID
}

type ItemPedido {
  productoId: ID!
  "Nombre y precio guardados al momento de la compra (historial estable)."
  nombre: String!
  cantidad: Int!
  precioUnitario: Int!
  subtotal: Int!
  "Estado actual del producto en inventory. Null si inventory no responde o el producto ya no existe."
  producto: ProductoActual
}
```

```
type ProductoActual {
  id: ID!
  nombre: String!
  tipo: String!
  imagen: String!
  disponible: Boolean!
  precioActual: Int!
}

type Entrega {
  nombreReceptor: String!
  calle: String!
  numero: String!
  depto: String
  comuna: String!
  region: String!
  telefono: String!
  indicaciones: String
}

type EventoPedido {
  estado: String!
  fecha: String!
}

type Pago {
  id: ID!
  estado: String!
  monto: Int!
  actualizadoEn: String!
  boleta: Boleta
}

type Boleta {
  folio: Int!
  neto: Int!
  iva: Int!
  total: Int!
  emitidaEn: String!
}

type PanelVentas {
  desde: String!
  hasta: String!
  totalVentas: Int!
  pedidosPagados: Int!
  ticketPromedio: Int!
  pedidosCancelados: Int!
  ventasPorDia: [VentaDia!]!
```

```
  ventasPorTipo: [VentaTipo!]!
  topProductos: [TopProducto!]!
  pedidosPorRegion: [PedidosRegion!]!
  reservasPorPaquete: [ReservasPaquete!]!
  "Desde inventory. Null si inventory no responde (error parcial)."
  stockBajo: [StockBajo!]
  generadoEn: String!
}

type VentaDia { dia: String!, ventas: Int!, pedidos: Int! }
type VentaTipo { tipo: String!, ventas: Int!, unidades: Int! }
type TopProducto { productoId: ID!, nombre: String!, unidades: Int!, ventas: Int! }
type PedidosRegion { region: String!, pedidos: Int! }
type ReservasPaquete { paquete: String!, reservas: Int!, personas: Int! }
type StockBajo { id: ID!, nombre: String!, stock: Int! }
```

8.3 Resolvers y controles

Resolver	Fuente	Control
Query.misPedidos	orders GET /internal/orders	requireRole(CLIENTE); tamano 1-10; orders filtra por el usuario verificado
Query.resumenPedido	orders GET /internal/orders/{id}	requireRole(CLIENTE, ADMIN); id validado; orders aplica propiedad (404 → null)
Query.panelVentas	analytics GET /internal/sales-summary	requireRole(ADMIN); fechas validadas; analytics vuelve a exigir ADMIN
ItemPedido.producto	DataLoader → inventory /internal/products/batch	Una llamada por solicitud (batching); null si el producto ya no existe
ResumenPedido.pago	DataLoader → payments /internal/ payments?pedidoIds=	Una llamada por página; payments filtra por cliente salvo ADMIN
ResumenPedido.clienteId	orders	FORBIDDEN_FIELD si el rol no es ADMIN (autorización por campo)
PanelVentas.stockBajo	inventory /internal/products/low-stock	inventory exige rol ADMIN propagado

- **Identidad verificada:** sólo acepta llamadas con credencial gateway→graphql y usuario propagado (prueba G07: llamada directa → 401).
- **Límites:** profundidad máxima 4 (la que usa el frontend; regla propia que soporta ciclos de fragmentos), costo máximo 2.500 (cuenta campos y multiplica por tamaño de página o listas, por lo que los alias repetidos se rechazan), sin introspección, sin lotes de operaciones, cuerpo $\leq$ 8 KB. Pruebas G02-G06.
- **Plazos y errores parciales:** cada llamada a un subsistema tiene 3 s. Si payments o inventory no responden, la consulta igual entrega los datos de orders con `pago: null` o `producto: null` y un error con código `DEPENDENCY_UNAVAILABLE` (prueba R01: respuesta en $<$ 6 s con payments detenido).
- Hallazgo durante las pruebas: la librería graphql-depth-limit respondía 500 ante un ciclo de fragmentos; se reemplazó por una regla propia que lo maneja (prueba G02).

8.4 Consulta de prueba y respuesta

```
query MisPedidos($pagina: Int!) {
  misPedidos(pagina: $pagina, tamano: 5) {
    total
    items { numero estado total despacho
      items { nombre cantidad subtotal producto { imagen disponible } }
      entrega { comuna region }
      pago { estado boleta { folio neto iva total } } }
  }
}
```


```
{"data":{"misPedidos":{"total":1,"items":[{"numero":152,"estado":"PAGADO","total":81800,"despacho":0,
  "items":[{"nombre":"Kuyen (blend tinto)","cantidad":1,"subtotal":32000,
            "producto":{"imagen":"images/vinos/kuyen-tinto.jpg","disponible":true}}, ...],
  "entrega":{"comuna":"Providencia","region":"Metropolitana"},
  "pago":{"estado":"APROBADO","boleta":{"folio":9,"neto":68739,"iva":13061,"total":81800}}}]}}}
```


8.5 Matriz de autorización GraphQL

Operación / campo	Visitante	CLIENTE	ADMIN	SOMMELIER
misPedidos	No (401 gateway)	Sí, sólo propios	No (FORBIDDEN)	No (FORBIDDEN)
resumenPedido	No	Sí, si es propio; si no null	Sí	No
ResumenPedido.clienteId	No	No (FORBIDDEN_FIELD)	Sí	No
ResumenPedido.entrega	No	Sí (propio)	Sí	No
panelVentas	No	No (FORBIDDEN)	Sí	No

8.6 Evidencia de la agregación ejecutada a través del API Gateway

Por indicación del docente la entrega no integra el frontend; la agregación se diseñó para la vista "Mis pedidos" (brecha B3) y el panel de reportes (B5) y se ejecuta a través del API Gateway con un cliente HTTP, exactamente como la invocaría el frontend: `POST /api/v1/graphql` con la cookie de sesión. Extracto real de la demostración (`docs/evidencias/demo_api/flujos.md`, paso 21):

```
POST /api/v1/graphql

{
 "query": "{ misPedidos(tamano: 3) { total items { numero estado total despacho items { nombre cantidad producto { imagen disponible } } pago { estado boleta { folio
    neto iva total } } } } }"
}
```


```
{
 "data": {
  "misPedidos": {
```

```
  "total": 1,
  "items": [
   {
    "numero": 1584,
    "estado": "PAGADO",
    "total": 81800,
    "despacho": 0,
    "items": [
     {
      "nombre": "Kuyen (blend tinto)",
      "cantidad": 1,
      "producto": {
       "imagen": "images/vinos/kuyen-tinto.jpg",
       "disponible": true
      }
     },
     {
      "nombre": "Cipreses Vineyard Sauvignon Blanc",
      "cantidad": 2,
      "producto": {
       "imagen": "images/vinos/cipreses-blanco.webp",
       "disponible": true
      }
     }
    ],
    "pago": {
     "estado": "APROBADO",
     "boleta": {
      "folio": 17,
      "neto": 68739,
      "iva": 13061,
      "total": 81800
     }
    }
   }
  ]
 }
}
```


Pasos 22 y 25 de la misma demostración: un cliente que pide `clienteId` recibe `FORBIDDEN_FIELD`, y otro cliente que consulta el pedido de Ana recibe `resumenPedido: null` sin datos. Paso 27: una llamada directa a `orders` sin credencial de servicio recibe 401; paso 28: la ruta interna no existe en el gateway (404). En Docker, además, los subsistemas no tienen puertos publicados (red backend `internal`). Pruebas automatizadas equivalentes: V4, V5 y G07.

Módulo 9 — Analítica y comunicación gRPC

Se caracteriza la carga proyectada, se documentan las decisiones de aplicabilidad de analítica y gRPC mediante ADR y se presentan los resultados de la prueba de carga ejecutada, comparados con los objetivos declarados.

9.1 Caracterización de la carga

Proyección (declarada como proyección, no medición): tienda boutique con más de 200 etiquetas y 7 degustaciones por semana. En un peak de campaña (Fiestas Patrias, Navidad) se proyectan ~50 visitantes simultáneos, ~5 solicitudes/s al catálogo, ~300 pedidos/día (≈1 pedido cada 30 s en horario punta) y < 100 reservas/semana. Cada pedido implica 3-4 llamadas orders→inventory (cotizar, reservar, confirmar o liberar). Los reportes se consultan pocas veces al día sobre algunos miles de eventos al mes.

Necesidad	Volumen / frecuencia	Tamaño de mensaje	Latencia esperada	Streaming	Decisión
Catálogo público	Alta (la lectura más frecuente)	≈4 KB	p95 < 300 ms	No	REST vía gateway
Precios y reservas de stock orders→inventory	Media, en ráfagas; crítica para no sobrevender	< 1 KB	p95 < 50 ms por llamada	No	**gRPC unary** (ADR-01)
Reportes de ventas	Baja (pocas consultas/día)	≈2 KB	p95 < 800 ms; retraso aceptado 1 min	No	**Servicio de analítica** separado, sin big data (ADR-02)
Seguimiento de pedido	Baja; el cliente consulta a demanda	≈3 KB	p95 < 800 ms	No: no hay eventos en tiempo real que empujar al navegador	GraphQL, sin streaming

9.2 ADR-01: gRPC entre orders e inventory

- **Contexto:** cada compra exige consultar precios oficiales y reservar stock de forma atómica e idempotente entre dos servicios; un error o una respuesta tardía puede sobrevender un vino de edición limitada (Chadwick tiene 4 unidades).
- **Alternativas:** (a) REST JSON interno; (b) gRPC unary; (c) gRPC streaming; (d) cola de mensajes asíncrona.
- **Elección:** gRPC unary con contrato `.proto` tipado, mTLS (identidad por certificado y autorización por método), plazos de 1 s (cotizar) y 2 s (reservar/confirmar/liberar), códigos de estado específicos (FAILED_PRECONDITION sin stock, INVALID_ARGUMENT, DEADLINE_EXCEEDED, PERMISSION_DENIED) e idempotencia por `reservation_id`. No se eligió streaming porque cada operación es una decisión corta y aislada, sin flujo continuo; tampoco una cola, porque el cliente necesita la confirmación de stock en la misma solicitud.
- **Consecuencias:** contrato fuerte y validado en ambos lados; mensajes acotados (64 KB entrada); el cliente se reconecta en ≤3 s si inventory se reinicia (prueba R07); se agrega la gestión de certificados. No se afirma mejora de rendimiento frente a REST: no se midió esa comparación; sí se midió que cumple su objetivo (C5).

9.3 ADR-02: servicio de analítica

- **Contexto:** la gerencia necesita ventas por día, por tipo, más vendidos, ticket promedio y reservas por paquete, sin cargar las bases transaccionales ni exponer datos personales.
- **Alternativas:** (a) consultas de reporte directas sobre orders; (b) servicio de analítica con eventos; (c) plataforma de big data.
- **Elección:** (b). orders y tastings publican eventos con patrón outbox (guardados junto al cambio de estado y entregados cada 2 s con reintentos); analytics los guarda deduplicados por eventId y calcula agregaciones con el pipeline de MongoDB. Retraso aceptado: 1 minuto. Permisos: sólo GraphQL con rol ADMIN. Datos personales: los eventos no llevan clienteId, nombre, correo ni dirección (sólo región) y analytics elimina esos campos si llegaran. Big data no se justifica: el volumen proyectado es de miles de eventos al mes.
- **Consecuencias:** reportes independientes de la disponibilidad de orders; consistencia eventual (prueba F13 verifica que una venta aparece en el panel en segundos).

9.4 Prueba de carga (medición)

Métricas reportadas: concurrencia (clientes simultáneos), throughput (solicitudes por segundo), latencias p50, p95 y p99 en milisegundos, porcentaje de errores y consumo de CPU y memoria RSS por proceso. El cumplimiento se evalúa comparando el p95 medido y la tasa de error con el objetivo definido antes de la medición.

Entorno: local (procesos, TLS en todos los tramos); 2 vCPU, 8.4 GB RAM, Python 3.13.18; todos los componentes en la misma máquina. Duración 30 s por escenario; datos ficticios del seed; script reproducible `tests/load_test.py` (resultados completos en `docs/evidencias/carga/resultados.json`). Para que una sola IP de prueba no fuera frenada por el límite de solicitudes se usó `RATE_LIMIT_MULTIPLIER=1000`; el límite real se prueba aparte.

Escenario	Concurrencia	Solicitudes	Throughput (sol/s)	p50 ms	p95 ms	p99 ms	Errores	Objetivo p95	Cumple
C1 Catálogo público (S01) vía gateway	20	6388	212.2	88.7	149.1	194.3	0 %	< 300 ms	Sí
C2 Cotizar carrito (S02) → gRPC QuoteItems	10	6401	213.2	45.5	68.5	83.1	0 %	< 400 ms	Sí
C3 Crear y anular pedido (S10+S13) → gRPC Reserve/Release	5	1547	51.3	93.6	138.9	173.8	0 %	< 800 ms	Sí
C4 GraphQL misPedidos (S11) orders+inventory+payments	5	2451	81.4	58.2	94.8	117.1	0 %	< 800 ms	Sí
C5 gRPC QuoteItems directo (orders→inventory, mTLS)	10	34882	1160.1	8.4	13.9	17.8	0 %	< 50 ms	Sí

Escenario	CPU promedio por proceso (máx.)	Memoria RSS máxima
C1	gateway 75.3 % (83.9 %) · inventory 45.3 % (51.6 %) · mongo 16.5 % (19.9 %) · vault 0.5 % (3 %)	gateway 71.9 MB · inventory 104 MB · mongo 224.5 MB · vault 236.6 MB
C2	gateway 67.1 % (80.9 %) · orders 44.1 % (49 %) · inventory 18 % (20 %) · mongo 7 % (9 %)	gateway 73.2 MB · orders 101.1 MB · inventory 104.5 MB · mongo 224.7 MB
C3	gateway 45.6 % (57.8 %) · orders 38.8 % (50 %) · auth 24 % (27 %) · mongo 22.5 % (25 %)	gateway 73.4 MB · orders 101.5 MB · auth 154.9 MB · mongo 233 MB

Escenario	CPU promedio por proceso (máx.)	Memoria RSS máxima
C4	gateway 37.5 % (43.9 %) · orders 27.8 % (40 %) · graphql 24.3 % (37.9 %) · mongo 20.2 % (23.9 %)	gateway 73.4 MB · orders 102.2 MB · graphql 182.4 MB · mongo 235.6 MB
C5	inventory 88 % (94.8 %) · mongo 26.2 % (31 %) · orders 6.1 % (22 %) · analytics 4.6 % (17 %)	inventory 100.1 MB · mongo 236.7 MB · orders 102.4 MB · analytics 75.4 MB

Lectura de resultados: todos los escenarios cumplen su objetivo con 0 % de errores y con concurrencias mayores a la proyectada. El cuello de botella es la CPU del gateway (incluye TLS y la introspección de sesión en cada solicitud) en una máquina de 2 vCPU compartida por todos los servicios. En C3, 1.547 pedidos se crearon y anularon bajo carga sin perder ni duplicar stock. Para mayor demanda el gateway es el primer candidato a replicar.

9.5 Pruebas gRPC y de analítica

Prueba	Qué demuestra	Resultado
R05 reserva idempotente y liberación	Repetir ReserveStock no descuenta dos veces; liberar dos veces devuelve una vez; no se confirma una reserva liberada	Aprobado
R06 validación y plazo	INVALID_ARGUMENT con cantidad 999; DEADLINE_EXCEEDED con plazo de 1 ms	Aprobado
R07 reinicio de inventory	El canal gRPC de orders se reconecta y vuelve a crear pedidos	Aprobado
V5 gRPC sin certificado / con certificado de payments	Handshake rechazado / PERMISSION_DENIED	Aprobado
F07 concurrencia (8 compras simultáneas, stock 3)	Exactamente 3 pedidos creados y 5 rechazados	Aprobado
F13 analítica	Una venta pagada aparece en el panel; sin datos personales en los eventos	Aprobado

Módulo 10 — Backend completo e integración verificable

Se describe la implementación entregada, la persistencia, el procedimiento de despliegue y operación, y la evidencia de verificación: pruebas automatizadas, matriz de trazabilidad requisito–pantalla–servicio–prueba y demostración de los flujos principales vía API.

10.1 Qué se implementó

- Todos los requisitos R01-R13 están implementados en 34 servicios. Por indicación del docente, la entrega es **backend con los datos del frontend**: las operaciones de las pantallas P01-P09 y de las brechas B1-B6 están disponibles en la API y se demuestran con un cliente HTTP (49 pasos) y 97 pruebas automatizadas; los datos iniciales salen del frontend v1.0 (`scripts/datos_frontend.json`).
- **Persistencia real:** MongoDB 8 con una base por servicio, índices creados al iniciar (únicos, parciales y TTL), catálogo y paquetes del frontend y usuarios ficticios cargados por `seed_data.py`. No hay respuestas fijas ni datos en memoria.
- **Integración:** API Gateway, Auth Service, Vault, GraphQL Gateway y siete microservicios privados con TLS en todos los tramos, mTLS en gRPC y propagación de identidad verificada.
- **Pagos:** adaptador de prueba (PagoSeguro Sandbox) que preserva el contrato de una pasarela real: credenciales del comercio, página de pago propia, notificación firmada, consulta servidor a servidor y transiciones. Limitación: el ambiente de integración de Transbank no fue accesible desde el entorno de pruebas (sin Internet); el adaptador `TransbankProvider` está implementado y se activa con `PAYMENT_PROVIDER=transbank`, pero no fue ejecutado. La boleta se emite internamente (folio, neto, IVA 19 %); el envío al SII queda como integración externa futura.
- **Mensajería:** mensajes de contacto persistidos y gestionados por ADMIN (S22, S32, S33); no se envían correos (no existe proveedor de correo en el caso).

10.2 Persistencia

Base	Colecciones	Índices relevantes
vu_auth	credentials, sessions	email único; TTL en sessions.exp
vu_clients	profiles, contact_messages	TTL 365 días en recibidoEn
vu_inventory	products, reservations, stock_movements	(tipo, activo)
vu_orders	orders, counters, outbox	(clienteId, idempotencyKey) único; (estado, venceEn)
vu_payments	payments, webhook_events, counters	tokenProveedor único; pedidoId único parcial para INICIADO y para APROBADO
vu_tastings	packages, sessions, reservations, outbox	(fecha, hora)
vu_analytics	events	(tipo, dia); _id = eventId
vu_sandbox	transactions	(proveedor simulado)

10.3 Despliegue y operación

La guía completa para instalar, ejecutar y probar el sistema paso a paso está en `Solemne_2/README.md`: requisitos por sistema operativo (Linux, Windows y macOS), inicio con Docker, usuarios de prueba, ejemplos de prueba con curl (catálogo, cotización, registro, compra, pago, GraphQL y comprobaciones de seguridad), uso de la colección Postman, pruebas automáticas, cómo detener el sistema y una tabla de problemas frecuentes. La tabla siguiente resume los comandos principales.

Acción	Comando
Preparar (una vez)	`python scripts/setup.py` → .env aleatorio, CA y certificados, carpetas de tokens
Iniciar	`docker compose up --build` → API en https://localhost:8443/api/v1
Detener	`docker compose down` (`-v` borra los datos)
Datos de prueba	Automáticos (vault-init ejecuta seed_data.py); credenciales en runtime/seed-out/
Pruebas	`docker compose --profile test run --rm tests` o, en modo local, `pytest tests -v`
Demostración vía API	`python tests/demo_api.py` (genera docs/evidencias/demo_api/flujos.md) o la colección `docs/postman/VinaUrbana.postman_collection.json`
Datos del frontend	`python scripts/extraer_datos_frontend.py` regenera scripts/datos_frontend.json desde Entrega_Frontend/index.html
Rotar un secreto	`docker compose run --rm vault-init python /scripts/vault_init.py --rotate internal/inventory/gateway`
Sin Docker	`python scripts/run_local.py [--test-profile]` y `python scripts/stop_local.py`
Health checks	`GET /health` en cada servicio (incluye ping a su base); HEALTHCHECK en cada imagen
Logs	JSON con request_id de extremo a extremo; sin cuerpos, tokens, contraseñas ni query strings

Dependencias versionadas: `backend/requirements.txt` (FastAPI 0.162.2, uvicorn 0.54.0, httpx 0.28.1, pymongo 4.19.0, argon2-cffi 25.1.0, PyJWT 2.15.0, grpcio 1.85.1) y `backend/gql-gateway/package.json` con `package-lock.json` (@apollo/server 5.5.1, express 5.3.0, graphql 16.15.0, dataloader 2.2.3). Imágenes: python:3.12-slim, node:22-alpine, mongo:8.0, hashicorp/vault:1.20.

Nota sobre la verificación: las pruebas se ejecutaron en modo local (mismos procesos, TLS y puertos que en Docker, sin contenedores) porque el entorno donde se construyó la entrega no tenía un demonio de Docker disponible. El archivo `docker-compose.yml` se validó con `docker compose config`; se recomienda ejecutar `docker compose up --build` y el perfil de pruebas antes de la presentación.

10.4 Pruebas automatizadas

Archivo	Cantidad	Cubre
test_datos_frontend.py	10	La API expone exactamente los vinos, precios, recomendados, paquetes, horarios, asuntos y regla de despacho del frontend v1.0
test_contracts.py	11	Contratos REST, filtros, despacho, errores controlados, 404/405, cabeceras de seguridad
test_security.py	47	Casos V1-V6, CSRF, cookie, login, registro, Argon2id, aislamiento de bases, políticas de Vault, 413, 429, logs
test_flows.py	14	Compra completa, idempotencia, webhook duplicado, doble pago, rechazo y reintento, anulación, concurrencia, despacho, reservas, baja de cuenta, analítica, vencimiento y pago tardío

Archivo	Cantidad	Cubre
test_graphql.py	8	Agregación de 3 subsistemas, ciclos de fragmentos, costo, paginación, introspección, batching, acceso directo
test_resilience.py	7	payments e inventory caídos, Vault sin respuesta, rotación de credencial, gRPC idempotente, plazos y reconexión
Total	**97 / 97 aprobadas**	Ejecución del 08-11-2026 sobre el sistema recién inicializado (57,8 s). Evidencia: docs/evidencias/pruebas/pytest_salida.txt y junit.xml

10.5 Matriz requisito | pantalla | servicio | prueba | resultado | evidencia

Requisito	Pantalla / brecha	Servicio	Prueba automatizada	Resultado	Evidencia (demo_api/flujos.md)
R01	P01, P08, P09	S03	test_registro_menor_de_edad_y_password_debil	Aprobado	demo paso 9
R02	P02	S01	D01, D02, test_S01_* (3)	Aprobado	demo pasos 2, 3, 5
R03	P03	S01	D03	Aprobado	demo paso 4
R04	P07	S02	test_S02_* (3)	Aprobado	demo paso 6
R05	P07, P09, B1	S02, S10	D07, test_S02_tarifas_de_despacho, F01, F02, F07	Aprobado	demo pasos 6, 7, 15, 16
R06	P07, P09, B1, B2	S14, S15, S16	F01, F03, F04, F05, F14, V6	Aprobado	demo pasos 14, 17-20
R07	P08, B3	S03-S09	V2, cookie, login, F11	Aprobado	demo pasos 8, 10-13, 48, 49
R08	P05, B3, B4	S17-S21, S30, S31	D04, D05, F09, F10	Aprobado	demo pasos 29-32, 46
R09	P06, B4	S22, S32, S33	D06, F12, límite 429	Aprobado	demo pasos 33, 43
R10	P04	— (estático)	No requiere backend	No aplica	—
R11	B4	S23-S29	F07, F08, V3	Aprobado	demo pasos 35-42, 47
R12	B3	S11, S12, S13, S15	V4, G01, F06, F08	Aprobado	demo pasos 21, 22, 25, 26
R13	B5	S34	F13, V3 panel	Aprobado	demo paso 44

10.6 Ejemplo: consulta protegida autorizada y denegada

Regla de autorización: "un cliente sólo consulta sus pedidos" aplicada por orders (`/internal/orders/{id}`: clienteId == usuario verificado). Ejecución autorizada (test_V4_dueno_consulta_su_pedido): el cliente A obtiene `resumenPedido.id`. Ejecución denegada con otra cuenta (test_V4_otro_cliente_no_ve_el_pedido): el cliente B recibe `resumenPedido: null` y la respuesta no contiene el nombre ni la comuna de A. Ambas pruebas pasan por el mismo gateway, el mismo resolver y el mismo servicio.

10.7 Demostración de los flujos principales

La demostración se hace vía API, sin frontend (indicación del docente). `tests/demo_api.py` recorre los flujos del caso a través del gateway y guarda cada solicitud y respuesta (sin cookies ni tokens) en `docs/evidencias/demo_api/flujos.md`. Para mostrarla en vivo se entrega además la colección `docs/postman/VinaUrbana.postman_collection.json` (29 solicitudes, guarda pedidoId y pagoId automáticamente). Resultado de la última ejecución: **49 de 49 pasos con el resultado esperado**.

#	Paso	Servicio	Esperado	Obtenido
1	Raíz informativa del gateway	—	200	200
2	Catálogo completo (9 vinos del frontend)	S01	200	200
3	Filtro Tintos (botón del frontend)	S01	200	200
4	Recomendados (carrusel del frontend)	S01	200	200
5	Filtro inválido	S01	400	400
6	Cotizar carrito: 1 Kuyen + 2 Cipreses a Providencia	S02	200	200
7	Cotizar a regiones (tarifa $6.990)	S02	200	200
8	Registro de cliente	S03	201	201
9	Registro de menor de edad (rechazado)	S03	422	422
10	Login con contraseña incorrecta	S04	401	401
11	Login correcto (sesión en cookie HttpOnly)	S04	200	200
12	Consultar sesión	S06	200	200
13	Guardar dirección en el perfil	S08	200	200
14	Pedido con total alterado por el cliente (rechazado)	S10	400	400
15	Crear pedido (stock reservado por gRPC)	S10	201	201
16	Reintento con la misma Idempotency-Key (mismo pedido)	S10	200	200
17	Iniciar pago (monto lo fija orders)	S14	201	201
18	Verificar pago con el proveedor (boleta emitida)	S15	200	200
19	Reenvío de la misma notificación del proveedor (sin efectos dobles)	S16	200	200
20	Intentar pagar de nuevo el mismo pedido	S14	409	409
21	Mis pedidos (una consulta, tres subsistemas)	S11	200	200
22	Campo restringido clienteId para un cliente	S11	200	200
23	Registro de un segundo cliente	S03	201	201
24	Login segundo cliente	S04	200	200
25	Otro cliente consulta el pedido de Ana (null, sin datos)	S12	200	200
26	Otro cliente intenta anular el pedido de Ana	S13	404	404
27	Acceso directo a orders sin identidad de servicio	interno	401	401
28	Ruta interna por el gateway (no publicada)	gateway	404	404
29	Paquetes del frontend	S17	200	200
30	Cupos por horario	S18	200	200
31	Reservar Maridaje Gourmet para 4	S19	201	201
32	Mis reservas	S20	200	200
33	Mensaje del formulario de contacto	S22	201	201
34	Login administrador	S04	200	200
35	Cliente intenta administrar productos (denegado)	S23	403	403
36	Inventario con stock	S23	200	200
37	Ingreso de bodega +6 Almaviva	S26	200	200
38	Pedidos pagados	S28	200	200
39	Despacho → EN_PREPARACION	S29	200	200
40	Despacho → DESPACHADO	S29	200	200
41	Despacho → ENTREGADO	S29	200	200

#	Paso	Servicio	Esperado	Obtenido
42	Transición inválida (ENTREGADO → EN_PREPARACION)	S29	409	409
43	Mensajes de contacto pendientes	S32	200	200
44	Panel de ventas GraphQL (analytics + inventory)	S34	200	200
45	Login sommelier	S04	200	200
46	Sommelier ve reservas con comentarios	S30	200	200
47	Sommelier intenta ver pedidos (denegado)	S28	403	403
48	Cerrar sesión	S05	200	200
49	Operación tras cerrar sesión	S07	401	401

Extracto: pago verificado con el proveedor y boleta (paso 18)

```
POST /api/v1/payments/bbcf4a29-3a0a-451f-bc3c-07d4e1864d58/verify
```


```
{
 "pagoId": "bbcf4a29-3a0a-451f-bc3c-07d4e1864d58",
 "pedidoId": "3b978c1d-acbc-4a24-a4d8-8ff38ce8a9fd",
 "estado": "APROBADO",
 "monto": 81800,
 "actualizadoEn": "2026-11-08T19:24:37.797000",
 "boleta": {
  "folio": 17,
  "neto": 68739,
  "iva": 13061,
  "total": 81800,
  "emitidaEn": "2026-11-08T19:24:37.799834+00:00"
 }
}
```


Extracto: el navegador intenta fijar el total (paso 14)

```
POST /api/v1/orders
Idempotency-Key: 21b3773b-e017-49c0-a238-e38bdfae1f28

{
 "items": [
  {
   "productoId": "kuyen-tinto",
   "cantidad": 1
  },
  {
   "productoId": "cipreses-blanco",
   "cantidad": 2
  }
```

```
 ],
 "entrega": {
  "nombreReceptor": "Ana Pérez",
  "calle": "Av. Providencia",
  "numero": "1500",
  "comuna": "Providencia",
  "region": "Metropolitana",
  "telefono": "+56955551234"
 },
 "total": 1
}
```

```
{
 "error": {
  "code": "VALIDATION_ERROR",
  "message": "Datos inválidos",
  "requestId": "07b4f610-bf5d-4471-98bc-c7761e7f1d8d",
  "details": [
   {
    "campo": "total",
    "motivo": "Extra inputs are not permitted"
...
```

Conclusiones

Resultados

Se diseñó, implementó y validó el backend completo del caso Viña Urbana a partir de los datos del frontend v1.0. La autoridad sobre precios, totales, stock y estado de los pagos reside exclusivamente en el servidor, y las reglas del negocio (venta a mayores de 18 años, tarifas y gratuidad de despacho, capacidad de la sala de cata, plazo de pago de 15 minutos, transiciones del pedido) se aplican y verifican en ejecución.

La arquitectura separa los componentes por naturaleza y propiedad de datos: un único punto de entrada con políticas explícitas, servicios transaccionales con bases independientes, un gateway de agregación sin capacidad de escritura y un servicio analítico desacoplado mediante eventos que no contienen datos personales.

Decisiones principales

- Un único API Gateway público con lista cerrada de rutas y una segunda verificación de identidad, rol y propiedad en cada servicio.
- Siete sistemas privados con propiedad exclusiva de su base de datos, reforzada por usuarios de MongoDB con permisos restringidos a su propia base.
- Credenciales independientes por tramo, almacenadas en Vault con políticas de mínimo privilegio y rotables sin interrupción del servicio.
- gRPC unary con mTLS restringido a la reserva de stock, donde se requiere un contrato tipado, plazos estrictos e idempotencia; analítica separada de las transacciones mediante el patrón outbox.
- Ausencia de secretos del cliente en el sistema: los datos de tarjeta se procesan únicamente en el proveedor de pagos y las contraseñas se almacenan con Argon2id.

Limitaciones y trabajo futuro

- La integración visual del frontend queda fuera del alcance por indicación del docente; las brechas B1-B6 se resuelven en el backend y se propone su incorporación a la interfaz.
- La validación de pagos se realizó con un proveedor de prueba propio; el adaptador para el ambiente de integración de Transbank está implementado, pero no fue ejecutado.
- La boleta se emite internamente (folio, neto e IVA); su envío al Servicio de Impuestos Internos queda como integración futura.
- Vault opera en modo desarrollo; un despliegue productivo requiere almacenamiento persistente, proceso de unseal y autenticación de servicios mediante AppRole o mecanismo equivalente.
- Las pruebas se ejecutaron en modo local, con los mismos servicios, cifrado y puertos que la definición de Docker; `docker-compose.yml` fue validado sintácticamente y debe ejecutarse en un equipo con Docker antes de la presentación.

Anexos

A. Archivos de la entrega

Ruta	**Contenido**
Solemne_2/README.md	Instrucciones de inicio, detención, pruebas y secretos
Solemne_2/docker-compose.yml	Despliegue reproducible con redes separadas
Solemne_2/backend/	Código de servicios, gateway, GraphQL, proto, Dockerfiles, requirements
Solemne_2/scripts/datos_frontend.json	Datos extraídos del frontend v1.0 (vinos, paquetes, opciones de formularios, reglas)
Solemne_2/docs/postman/	Colección Postman para demostrar la API
Solemne_2/scripts/	setup.py, extraer_datos_frontend.py, vault_init.py, seed_data.py, run_local.py, stop_local.py, generar_coleccion_postman.py
Solemne_2/tests/	pytest (97 pruebas), demo_api.py, load_test.py
Solemne_2/docs/diagramas/	uml_dominio, uml_servicios, uml_gateway (.drawio y .mmd), mapa, secuencias y estados (.mmd), PNG
Solemne_2/docs/evidencias/	pruebas/ (salida, junit.xml), demo_api/ (flujos.md, resumen.json), carga/ (resultados.json, salida)

B. Referencias

- OWASP REST Security, Authorization y Secrets Management Cheat Sheets.
- Documentación oficial de GraphQL (seguridad), gRPC (conceptos, deadlines, control de flujo), HashiCorp Vault (KV v2, políticas) y MongoDB (roles, índices TTL).
- Material del curso: Microservicios.drawio, Integracion.drawio, apigateway.zip y las entregas previas del repositorio (Semanas 7-9).